Anti-Bluff Mandate Reinforcement — Group B Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use `superpowers:subagent-driven-development` (recommended, per Group A-prime pattern) or `superpowers:executing-plans` to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Tighten the matrix-runner gate with five reliability + observability fixes (per-row diagnostics, Teardown force-kill fast-path, pack-dir convention, JUnit XML failure parsing, concurrent-mode opt-in with gating-flip) — none relax existing clauses; all close the next class of bluffs the gate cannot yet detect.

Architecture: Components A-E live in `submodules/containers/pkg/emulator/` + `cmd/emulator-matrix/` (matrix-runner reliability + evidence — generic, reusable across `vasic-digital/*` consumers). Components F-H live in Lava parent (`scripts/run-emulator-tests.sh`, `scripts/tag.sh`, `tests/tag-helper/`, evidence files — Lava-domain version-string detection + tag-time gates). Three commits: 1 on Containers branch `lava-pin/2026-05-06-group-b`, then 2 on Lava `master` (parent code + pin bump/closure).

Tech Stack: Go 1.24+ (`encoding/xml`, `os/exec`, `syscall`, `sync`, stdlib `testing`); bash 5+ (`jq`); existing pre-push hook from Group A-prime; existing `pkg/emulator` package as the integration baseline.

File Structure

File	Responsibility
`submodules/containers/pkg/emulator/cleanup.go`	+ `KillByPort(ctx, port)` strict-adjace proc walk; reuses `procWalker` + `killer` seams
`submodules/containers/pkg/emulator/cleanup_test.go`	+ 4 KillByPort tests (no-match, strict-adjacent, substring-safety, SIGKILL-aft grace)
`submodules/containers/pkg/emulator/types.go`	

File	Responsibility
	+ `DiagnosticInfo`, `KillReport`, extend `BootResult.ConsolePort` already exists `TestResult` gains `Diag` + `FailureSummaries` + `Concurrent` fields; `MatrixResult.Gating` field
`submodules/containers/pkg/emulator/android.go`	~ `Teardown` post-30s-grace fast-path: invoke `KillByPort(port)`; skip-on-mismatch returns original timeout erro
`submodules/containers/pkg/emulator/android_test.go`	+ 2 Teardown fast-path tests (success-after-grace, skip-on-mismatch)
`submodules/containers/pkg/emulator/matrix.go`	~ `RunMatrix`: capture diag pre-test, pars JUnit XML post-test → `FailureSummari` + worker pool when `Concurrent > 1`; + `parseJUnitFailures` helper; ~ `writeAttestation` writes new fields including `gating`
`submodules/containers/pkg/emulator/matrix_test.go`	+ 7 tests (parseJUnitFailures: pass/sing failure/single-error/multi-suite/ malformed; gating defaults true; gating false on concurrent>1; gating false on dev)
`submodules/containers/cmd/emulator-matrix/main.go`	+ `--concurrent N` (default 1) and `--dev` (bool) flags; pipe through to `MatrixCon`
`scripts/run-emulator-tests.sh` (Lava)	~ auto-detect `versionName`/`versionCod` from `app/build.gradle.kts`; + `--tag <tag>` flag; + `--concurrent N` and `--de` passthrough; new evidence-dir convention
`scripts/tag.sh` (Lava)	~ `require_matrix_attestation_clause_` — add 3 new gates: reject `concurrent` 1 on any row, reject run-level `gating: false`, assert `diag.sdk == api_level` p row
`tests/tag-helper/test_tag_rejects_concurrent_attestation.sh`	+ Bash fixture test exercising tag.sh against synthetic attestation with `concurrent: 4` row
`tests/tag-helper/test_tag_rejects_non_gating_attestation.sh`	+ Bash fixture test exercising tag.sh against synthetic attestation with `gating: false`
`tests/tag-helper/test_tag_rejects_diag_sdk_mismatch.sh`	

File	Responsibility
	+ Bash fixture test exercising tag.sh against synthetic attestation with `diag.sdk=33` on an `api_level=34` row
`tests/tag-helper/test_tag_accepts_gating_serial_attestation.sh`	+ Bash fixture test exercising tag.sh against clean serial attestation (all 3 ga green)
`tests/tag-helper/run_all.sh`	+ Test runner that executes all 4 tag-helper tests and exits non-zero if any fa
`CLAUDE.md` (Lava root)	~ §6.I clause 4 documentation extensio row schema gains `diag` + `failure_summaries` + `concurrent`; run-level `gating` field documented
`submodules/containers` (gitlink)	~ Pin bump to lava-pin/2026-05-06-group-b HEAD
`.lava-ci-evidence/bluff-hunt/2026-05-06-group-b.json`	+ §6.N.1.1-style hunt (1-2 production f from gate-shaping surface)
`.lava-ci-evidence/Phase-Group-B-closure-2026-05-06.json`	+ Closure attestation: per-component SHAs, 5 mutation rehearsals, mirror convergence

Mirror & branch policy (read first)

- **Containers branch:** `lava-pin/2026-05-06-group-b` (NEW; cut from current `lava-pin/2026-05-05-clause-6n-prime` HEAD which is what the parent currently pins to). Push the branch to all 4 Containers upstreams (`github gitlab gitflic gitverse`) at the end of Phase A.
- **Convergence verification:** ALWAYS use live `git ls-remote <remote> <ref>` — NEVER read `.git/refs/remotes/<r>/<branch>`. The local cache is stale until you `git fetch`, and Group A-prime's reviewer iterations recorded multiple false-positive divergence findings from misreading it.
- **Push order:** `github` first (typically fastest + has the broadest tooling), then `gitlab`, `gitflic`, `gitverse`. After all four return, run `for r in github gitlab gitflic gitverse; do git ls-remote $r refs/heads/<branch>; done` and confirm the 4 SHAs match.

Phase A — Containers code (1 commit on `lava-pin/2026-05-06-group-b`)

> Working tree: `submodules/containers/`. All `git` commands in this phase are run from inside that directory. Tests run via `go test ./pkg/emulator/... -count=1 -race`.

Task A0: Branch setup

Files:

- (no edits, just branch creation)

☐ **Step 1: Verify clean working tree in the submodule**

```
cd submodules/containers
git status
```

Expected: `On branch lava-pin/2026-05-05-clause-6n-prime` (or whatever the current parent pin points to), `nothing to commit, working tree clean`.

☐ **Step 2: Cut the new Group B branch from the current HEAD**

```
cd submodules/containers
current_head=$(git rev-parse HEAD)
echo "Cutting lava-pin/2026-05-06-group-b from $current_head"
git checkout -b lava-pin/2026-05-06-group-b
```

Expected: `Switched to a new branch 'lava-pin/2026-05-06-group-b'.`

☐ **Step 3: Sanity-check the package compiles + tests pass on the new branch BEFORE any change**

```
cd submodules/containers
go build ./pkg/emulator/... ./cmd/emulator-matrix/...
go test ./pkg/emulator/... -count=1 -race
```

Expected: build succeeds; all existing tests pass.

If anything fails here, STOP — Group A-prime end state is corrupt and must be fixed before Group B begins.

Task A1: KillByPort — failing tests first (TDD)

Files:

- Modify: `submodules/containers/pkg/emulator/cleanup_test.go`

☐ **Step 1: Add 4 KillByPort tests (all expected to fail because the function does not exist yet)**

Append to `submodules/containers/pkg/emulator/cleanup_test.go`:

```
// ---------------------------------------------------------------------
// KillByPort tests — Group B clause 6.I extension
//
// Forensic anchor: the matrix-runner Teardown's `adb emu kill`
        retains
// its 30s grace, but stuck QEMU instances persist past it (clause
// 6.M-recorded behavior). Without a port-strict force-kill fast-
        path,
// the next iteration's Boot lands on 5556/5557 and the matrix
// accumulates concurrent emulators — observed flakes in the API
        35 row
// of the 5-AVD matrix.
//
// SAFETY contract for KillByPort (the tests below verify each
        clause):
//   - Strict adjacent token match — substring `25554` must NOT
        match port 5554
//   - No-op on mismatch — concurrent emulators on other ports
        untouched
//   - SIGKILL only after 5s grace — graceful exit honored first
// ---------------------------------------------------------------------

type fakeProcWalkerWithCmdlines struct {
    cmdlines map[int][]string // pid → argv
}

func (f fakeProcWalkerWithCmdlines) PidComms() (map[int]string,
        error) {
    out := make(map[int]string)
    for pid, argv := range f.cmdlines {
        if len(argv) > 0 {
            out[pid] = argv[0]
        } else {
            out[pid] = ""
        }
    }
    return out, nil
}

func (f fakeProcWalkerWithCmdlines) PidCmdlines() (map[int]
        []string, error) {
```

```
    return f.cmdlines, nil
}

type fakeKillerByPort struct {
    signaled  map[int][]syscall.Signal
    aliveAfter map[syscall.Signal]map[int]bool // post-signal
        aliveness override
}

func newFakeKillerByPort() *fakeKillerByPort {
    return &fakeKillerByPort{
        signaled:   make(map[int][]syscall.Signal),
        aliveAfter: make(map[syscall.Signal]map[int]bool),
    }
}

func (f *fakeKillerByPort) Signal(pid int, sig syscall.Signal)
        error {
    f.signaled[pid] = append(f.signaled[pid], sig)
    return nil
}

func (f *fakeKillerByPort) Exists(pid int) bool {
    // Default: SIGTERM clears the process; SIGKILL clears it.
    // Tests override aliveAfter[sig][pid] = true to keep the
        process
    // alive past the corresponding signal (forces SIGKILL path).
    signals, ok := f.signaled[pid]
    if !ok {
        return true // never signaled → still alive
    }
    last := signals[len(signals)-1]
    if alive, override := f.aliveAfter[last][pid]; override {
        return alive
    }
    return false
}

func TestKillByPort_NoMatch_NoOp(t *testing.T) {
    w := fakeProcWalkerWithCmdlines{cmdlines: map[int][]string{
        1234: {"qemu-system-x86_64", "-avd", "Pixel_9a", "-port",
        "5556"},
        5678: {"chrome", "--user-data-dir=/tmp/x"},
    }}
    k := newFakeKillerByPort()
    report, err := killByPortWithDeps(context.Background(), 5554,
        w, k)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    if report.Matched != 0 {
        t.Fatalf("expected Matched=0 (no proc has -port 5554
        adjacent), got %d", report.Matched)
    }
```

```
    if len(k.signaled) != 0 {
        t.Fatalf("expected no kill signals issued, got %v",
        k.signaled)
    }
}

func TestKillByPort_StrictAdjacentMatch(t *testing.T) {
    w := fakeProcWalkerWithCmdlines{cmdlines: map[int][]string{
        1111: {"qemu-system-x86_64", "-avd", "A1", "-port",
        "5554"}, // MATCH
        2222: {"qemu-system-x86_64", "-avd", "A2", "-port",
        "5556"}, // no match
    }}
    k := newFakeKillerByPort()
    report, err := killByPortWithDeps(context.Background(), 5554,
        w, k)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    if report.Matched != 1 {
        t.Fatalf("expected Matched=1, got %d", report.Matched)
    }
    if len(report.Sigtermed) != 1 || report.Sigtermed[0] != 1111 {
        t.Fatalf("expected Sigtermed=[1111], got %v",
        report.Sigtermed)
    }
    if _, signaled := k.signaled[2222]; signaled {
        t.Fatalf("pid 2222 was signaled despite different port —
        safety violation")
    }
}

func TestKillByPort_SubstringSafety(t *testing.T) {
    // pid 9999 has the literal string "5554" inside its argv but
        NOT
    // adjacent to "-port". KillByPort(5554) MUST NOT match it.
    w := fakeProcWalkerWithCmdlines{cmdlines: map[int][]string{
        9999: {"qemu-system-x86_64", "-avd", "A1", "-port",
        "25554"}, // 25554 ≠ 5554
        8888: {"qemu-system-x86_64", "-avd", "A2", "-pidfile", "/
        tmp/5554.pid"},
    }}
    k := newFakeKillerByPort()
    report, err := killByPortWithDeps(context.Background(), 5554,
        w, k)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    if report.Matched != 0 {

        t.Fatalf("expected Matched=0 (no adjacent token pair), got
        %d (signaled=%v)",
            report.Matched, k.signaled)
    }
```

```
}

func TestKillByPort_RequiresSIGKILL_AfterGrace(t *testing.T) {
    w := fakeProcWalkerWithCmdlines{cmdlines: map[int][]string{
        7777: {"qemu-system-x86_64", "-avd", "A1", "-port",
        "5554"},
    }}
    k := newFakeKillerByPort()
    // Make pid 7777 survive SIGTERM — forces the SIGKILL grace
        path.
    // After SIGKILL, default Exists() returns false, so the
        process
    // is reported as Sigkilled, not Surviving.
    k.aliveAfter[syscall.SIGTERM] = map[int]bool{7777: true}
    report, err := killByPortWithDeps(context.Background(), 5554,
        w, k)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    if report.Matched != 1 {
        t.Fatalf("expected Matched=1, got %d", report.Matched)
    }
    if len(report.Sigkilled) != 1 || report.Sigkilled[0] != 7777 {
        t.Fatalf("expected Sigkilled=[7777], got %v",
        report.Sigkilled)
    }
}
```

- [ ] **Step 2: Run tests, confirm all 4 fail with "killByPortWithDeps undefined" or "PidCmdlines undefined"**

```
cd submodules/containers
go test ./pkg/emulator/... -run 'TestKillByPort' -count=1
```

Expected: compile error citing `killByPortWithDeps` and/or `PidCmdlines` undefined.

If tests fail to compile for a different reason, fix the test file before proceeding.

Task A2: KillByPort — minimal implementation

Files:

- Modify: `submodules/containers/pkg/emulator/cleanup.go`

- [ ] **Step 1: Extend the procWalker interface to expose argv (NUL-separated cmdline)**

In `submodules/containers/pkg/emulator/cleanup.go`, replace the `procWalker` interface and `osProcWalker` impl with the extended versions. Find this block (around lines 23-52):

```
// procWalker abstracts /proc enumeration so cleanup_test.go can
        inject
// synthetic /proc data. Production walks the real /proc.
type procWalker interface {
    PidComms() (map[int]string, error)
}

type osProcWalker struct{}

func (osProcWalker) PidComms() (map[int]string, error) {
    entries, err := os.ReadDir("/proc")
    if err != nil {
        return nil, fmt.Errorf("read /proc: %w", err)
    }
    out := make(map[int]string)
    for _, e := range entries {
        pid, err := strconv.Atoi(e.Name())
        if err != nil || pid <= 0 {
            continue
        }
        commPath := filepath.Join("/proc", e.Name(), "comm")
        b, err := os.ReadFile(commPath)
        if err != nil {
            // Best-effort: process may have exited mid-walk
            out[pid] = ""
            continue
        }
        out[pid] = strings.TrimSpace(string(b))
    }
    return out, nil
}
```

Replace with:

```
// procWalker abstracts /proc enumeration so cleanup_test.go can
        inject
// synthetic /proc data. Production walks the real /proc.
//
// PidComms returns pid → /proc/<pid>/comm (process name only,
        used by
// Cleanup's prefix matcher). PidCmdlines returns pid → argv slice
// (split on NUL bytes, used by KillByPort's strict adjacent-token
// matcher).
type procWalker interface {
    PidComms() (map[int]string, error)
    PidCmdlines() (map[int][]string, error)
}

type osProcWalker struct{}
```

```go
func (osProcWalker) PidComms() (map[int]string, error) {
    entries, err := os.ReadDir("/proc")
    if err != nil {
        return nil, fmt.Errorf("read /proc: %w", err)
    }
    out := make(map[int]string)
    for _, e := range entries {
        pid, err := strconv.Atoi(e.Name())
        if err != nil || pid <= 0 {
            continue
        }
        commPath := filepath.Join("/proc", e.Name(), "comm")
        b, err := os.ReadFile(commPath)
        if err != nil {
            // Best-effort: process may have exited mid-walk
            out[pid] = ""
            continue
        }
        out[pid] = strings.TrimSpace(string(b))
    }
    return out, nil
}

func (osProcWalker) PidCmdlines() (map[int][]string, error) {
    entries, err := os.ReadDir("/proc")
    if err != nil {
        return nil, fmt.Errorf("read /proc: %w", err)
    }
    out := make(map[int][]string)
    for _, e := range entries {
        pid, err := strconv.Atoi(e.Name())
        if err != nil || pid <= 0 {
            continue
        }
        cmdPath := filepath.Join("/proc", e.Name(), "cmdline")
        b, err := os.ReadFile(cmdPath)
        if err != nil {
            // Best-effort: process may have exited mid-walk;
        record
            // empty argv so the matcher simply skips it.
            out[pid] = nil
            continue
        }
        // /proc/<pid>/cmdline is NUL-separated. Trailing NUL is
        common.
        raw := strings.TrimRight(string(b), "\x00")
        if raw == "" {
            out[pid] = nil
            continue
        }
        out[pid] = strings.Split(raw, "\x00")
    }
```

```
    return out, nil
}
```

- [ ] **Step 2: Add `KillReport` struct and `KillByPort` + `killByPortWithDeps` functions**

Append to `submodules/containers/pkg/emulator/cleanup.go` (after the existing `cleanupWithDeps` function, before EOF):

```
// KillReport summarises the outcome of a KillByPort invocation.
//
// The Matched count is the gate the caller (Teardown fast-path)
        uses
// to decide whether the kill succeeded enough to short-circuit
        the
// "emulator did not exit" error path. Matched=0 is a no-op safe
        state:
// it means no /proc entry passed the strict adjacent-token check,
        and
// the caller MUST treat that as "fast-path skipped" — typically
        by
// returning the original timeout error so the matrix runner
        records
// an honest row failure.
type KillReport struct {
    // Matched is the number of /proc entries whose cmdline
        contained
    // "-port <port>" as adjacent argv tokens.
    Matched int
    // Sigtermed lists the PIDs that received SIGTERM (every
        matched
    // PID receives one).
    Sigtermed []int
    // Sigkilled lists the PIDs that survived the 5-second SIGTERM
    // grace and therefore received SIGKILL.
    Sigkilled []int
    // Surviving lists PIDs still alive after SIGKILL (rare;
        caused
    // by permission errors, kernel-level holds, or PID-reuse
        races).
    Surviving []int
}

// KillByPort attempts to terminate the process(es) whose cmdline
// contains "-port <port>" as adjacent argv tokens. Used by the
// matrix-runner's Teardown fast-path: when `adb emu kill` returns
// successfully but the underlying QEMU instance is stuck past the
// 30s grace, this function targets that specific QEMU child by
        its
// console port and SIGTERMs it (then SIGKILLs after a 5s grace).
//
// SAFETY (THE LOAD-BEARING INVARIANT):
//
```

```
//   - The match is STRICT adjacent-token. A process whose cmdline
//     contains "-port" "5554" as adjacent argv tokens MATCHES.
//     A process whose cmdline contains "-port" "25554" does NOT
       match
//     for port 5554 — the substring is irrelevant; tokenization
       is.
//   - On Matched=0, KillByPort is a complete no-op — no signals
       are
//     sent. Concurrent emulators on other ports, sibling vasic-
       digital
//     project QEMUs, and developer-spawned QEMUs are NEVER
       touched.
//
// This is the constitutional fix for the 2026-05-04 evening
       operator
// concern that "any broader pkill against session processes" is a
// Forbidden Command List violation. KillByPort is permitted
       because
// it targets a single, provably-this-matrix QEMU child by its
       argv,
// not by name.
//
// Bluff-Audit (recorded in the implementing commit body):
//
//  Mutation: weaken the matcher to strings.Contains(cmdline,
//            "-port "+strconv.Itoa(port))
//  Observed: TestKillByPort_SubstringSafety asserts that pid 9999
//            (whose argv contains "25554") is NOT matched for
       port
//            5554; the weakened matcher matches it because
       "25554"
//            contains "5554" as a substring.
//  Reverted: yes
func KillByPort(ctx context.Context, port int) (KillReport,
       error) {
    return killByPortWithDeps(ctx, port, osProcWalker{},
       osKiller{})
}

// killByPortWithDeps is the testable core; production KillByPort
       wires
// the real procWalker + killer.
func killByPortWithDeps(
    ctx context.Context,
    port int,
    w procWalker,
    k killer,
) (KillReport, error) {
    var report KillReport
    cmdlines, err := w.PidCmdlines()
    if err != nil {
        return report, err
    }
    target := strconv.Itoa(port)
```

```
for pid, argv := range cmdlines {
    // STRICT adjacent-token match. Walk argv looking for the
    // literal token "-port" immediately followed by the
    literal
    // port number. Substring matches and "-port=5554" forms
    are
    // intentionally NOT honoured — qemu-system never emits
    the
    // "=" form, and substring matches are the bluff vector
    this
    // API exists to prevent.
    matched := false
    for i := 0; i < len(argv)-1; i++ {
        if argv[i] == "-port" && argv[i+1] == target {
            matched = true
            break
        }
    }
    if matched {
        report.Matched++
        report.Sigtermed = append(report.Sigtermed, pid)
    }
    _ = pid // silence linters in case the loop never matches
}
if report.Matched == 0 {
    return report, nil
}
for _, pid := range report.Sigtermed {
    _ = k.Signal(pid, syscall.SIGTERM)
}
// Wait up to 5 seconds, polling every 250ms.
deadline := time.Now().Add(5 * time.Second)
var stragglers []int
for time.Now().Before(deadline) {
    select {
    case <-ctx.Done():
        return report, ctx.Err()
    case <-time.After(250 * time.Millisecond):
    }
    stragglers = stragglers[:0]
    for _, pid := range report.Sigtermed {
        if k.Exists(pid) {
            stragglers = append(stragglers, pid)
        }
    }
    if len(stragglers) == 0 {
        break
    }
}
for _, pid := range stragglers {
    if err := k.Signal(pid, syscall.SIGKILL); err == nil {
        report.Sigkilled = append(report.Sigkilled, pid)
    } else {
```

```
            report.Surviving = append(report.Surviving, pid)
        }
    }
    return report, nil
}
```

- [ ] **Step 3: Run KillByPort tests, confirm all 4 pass**

```
cd submodules/containers
go test ./pkg/emulator/... -run 'TestKillByPort' -count=1 -v
```

Expected: `--- PASS: TestKillByPort_NoMatch_NoOp`, `--- PASS: TestKillByPort_StrictAdjacentMatch`, `--- PASS: TestKillByPort_SubstringSafety`, `--- PASS: TestKillByPort_RequiresSIGKILL_AfterGrace`. PASS overall.

- [ ] **Step 4: Run the entire emulator package test suite — confirm Cleanup tests still pass after the procWalker interface extension**

```
cd submodules/containers
go test ./pkg/emulator/... -count=1 -race
```

Expected: every test passes; no race detected.

- [ ] **Step 5: Falsifiability rehearsal — record observed failure**

Temporarily weaken the matcher in `cleanup.go` to use a substring search. Replace the inner adjacent-token loop:

```
// MUTATION (will revert) — substring match
matched := false
for _, tok := range argv {
    if strings.Contains(tok, target) {
        matched = true
        break
    }
}
```

Run the test:

```
cd submodules/containers
go test ./pkg/emulator/... -run TestKillByPort_SubstringSafety
        -count=1 -v
```

Expected: `FAIL: TestKillByPort_SubstringSafety` — assertion message says `expected Matched=0 (no adjacent token pair), got 2 (signaled=...)`.

Save the observed failure verbatim — it goes into the commit body Bluff-Audit stamp.

- [ ] **Step 6: Revert the mutation, confirm tests pass again**

Restore the original adjacent-token loop. Run:

```
cd submodules/containers
go test ./pkg/emulator/... -run TestKillByPort -count=1 -v
```

Expected: PASS for all 4 KillByPort tests.

Task A3: Teardown fast-path — failing tests first

Files:

- Modify: `submodules/containers/pkg/emulator/android_test.go`

- [ ] **Step 1: Add 2 Teardown fast-path tests**

Append to `submodules/containers/pkg/emulator/android_test.go`:

```
// ---------------------------------------------------------------------
// Teardown fast-path tests — Group B
//
// After the existing 30s `adb emu kill` grace expires, Teardown
        invokes
// emulator.KillByPort(consolePort) to attempt a port-strict
        force-kill.
// On match, Teardown returns nil (the emulator was stuck but is
        now
// gone). On mismatch (Matched=0), Teardown returns the original
// "emulator did not exit" error — concurrent emulators on other
        ports
// are untouched.
//
// SAFETY: these tests verify the skip-on-mismatch invariant. The
// production code MUST NOT broaden the kill criterion to any
        process
// that "looks like" a stuck emulator.
// ---------------------------------------------------------------------

// fakeAdbExecutorAlwaysAlive is a CommandExecutor whose Execute()
        returns
// `adb devices` output that always includes the target
        localhost:<port>
// in "device" state — simulating a stuck emulator that ignores
        `adb emu
// kill`.
```

```
type fakeAdbExecutorAlwaysAlive struct {
    port int
}

func (f fakeAdbExecutorAlwaysAlive) Execute(_ context.Context,
        name string, args ...string) ([]byte, error) {
    // adb -s localhost:<port> emu kill — pretend it succeeds
    if len(args) >= 4 && args[2] == "emu" && args[3] == "kill" {
        return []byte("OK: killing emulator, bye bye\n"), nil
    }
    // adb devices — always report localhost:<port> as alive
    if len(args) == 1 && args[0] == "devices" {
        return []byte(fmt.Sprintf("List of devices
        attached\nlocalhost:%d\tdevice\n", f.port)), nil
    }
    return nil, nil
}

func (f fakeAdbExecutorAlwaysAlive) Start(_ context.Context, name
        string, args ...string) error {
    return nil
}

func TestTeardown_FastPath_SkipsOnMismatch(t *testing.T) {
    // Save and replace package-level KillByPort hook so the test
        is
    // hermetic. The production implementation walks /proc.
    prev := killByPortHook
    killByPortHook = func(_ context.Context, _ int) (KillReport,
        error) {
        // Mismatch: no /proc entry has -port <port> adjacent.
        return KillReport{Matched: 0}, nil
    }
    defer func() { killByPortHook = prev }()

    // Use a short test-only Teardown timeout so the 30s grace is
    // compressed for the test.
    prevGrace := teardownGracePeriod
    teardownGracePeriod = 200 * time.Millisecond
    defer func() { teardownGracePeriod = prevGrace }()

    a := NewAndroidEmulatorWithExecutor("/opt/android-sdk",
        fakeAdbExecutorAlwaysAlive{port: 5554})
    err := a.Teardown(context.Background(), 5554)
    if err == nil {
        t.Fatalf("expected Teardown to return an error when
        KillByPort.Matched==0 and emulator persists, got nil")
    }
    if !strings.Contains(err.Error(), "did not exit") {
        t.Fatalf("expected error to mention 'did not exit', got:
        %v", err)
    }
}
```

```
// fakeAdbExecutorStuckThenGone reports the target as alive on the
        first
// `adb devices` call and gone on subsequent calls — simulating a
        stuck
// emulator that the KillByPort fast-path successfully clears.
type fakeAdbExecutorStuckThenGone struct {
    port  int
    calls int
}

func (f *fakeAdbExecutorStuckThenGone) Execute(_ context.Context,
        _ string, args ...string) ([]byte, error) {
    if len(args) >= 4 && args[2] == "emu" && args[3] == "kill" {
        return []byte("OK: killing emulator, bye bye\n"), nil
    }
    if len(args) == 1 && args[0] == "devices" {
        f.calls++
        if f.calls <= 2 {
            return []byte(fmt.Sprintf("List of devices
        attached\nlocalhost:%d\tdevice\n", f.port)), nil
        }
        return []byte("List of devices attached\n"), nil
    }
    return nil, nil
}

func (f *fakeAdbExecutorStuckThenGone) Start(_ context.Context, _
        string, _ ...string) error {
    return nil
}

func TestTeardown_FastPath_SucceedsAfterKillByPort(t *testing.T) {
    prev := killByPortHook
    killByPortHook = func(_ context.Context, _ int) (KillReport,
        error) {
        return KillReport{Matched: 1, Sigtermed: []int{12345}},
        nil
    }
    defer func() { killByPortHook = prev }()
    prevGrace := teardownGracePeriod
    teardownGracePeriod = 200 * time.Millisecond
    defer func() { teardownGracePeriod = prevGrace }()

    a := NewAndroidEmulatorWithExecutor("/opt/android-sdk",
        &fakeAdbExecutorStuckThenGone{port: 5554})
    err := a.Teardown(context.Background(), 5554)
    if err != nil {
        t.Fatalf("expected Teardown to succeed after KillByPort
        cleared the stuck emulator, got: %v", err)
    }
}
```

Step 2: Run tests, confirm both fail with "killByPortHook undefined" or "teardownGracePeriod undefined"

```
cd submodules/containers
go test ./pkg/emulator/... -run 'TestTeardown_FastPath' -count=1
```

Expected: compile error citing the two undefined identifiers.

Task A4: Teardown fast-path — implementation

Files:

- Modify: `submodules/containers/pkg/emulator/android.go`

☐ **Step 1: Add the two test seams (`killByPortHook`, `teardownGracePeriod`) at file scope**

In `submodules/containers/pkg/emulator/android.go`, immediately after the `import` block (around line 11), add:

```
// killByPortHook is the package-level seam tests use to substitute a
// fake KillByPort implementation. Production Teardown uses the real
// KillByPort; tests override this so they don't have to spawn real
// QEMU processes to test the fast-path branch.
var killByPortHook = KillByPort

// teardownGracePeriod is the wall-clock time Teardown waits after
// `adb emu kill` before invoking the KillByPort fast-path. Set short
// in tests so the suite stays fast; defaults to 30 seconds in
// production (matches the 2026-05-05 grace already in the file).
var teardownGracePeriod = 30 * time.Second
```

☐ **Step 2: Refactor `Teardown` to honour `teardownGracePeriod` and invoke the fast-path on timeout**

Replace the `Teardown` function (currently around lines 394-438 of `android.go`):

```
func (a *AndroidEmulator) Teardown(ctx context.Context, port int) error {
    target := fmt.Sprintf("localhost:%d", port)
    out, err := a.executor.Execute(
        ctx, a.adbBinary(), "-s", target, "emu", "kill",
    )
    if err != nil {
```

```
    return fmt.Errorf("adb emu kill failed: %w; output=%s",
    err, out)
}

// Poll for the emulator to actually exit. Bound by
    teardownGracePeriod
// (30s in production; tests override to keep the suite fast).
    "Exit"
// means: the localhost:<port> entry is no longer in `adb
    devices`
// output as "device" (it may briefly show "offline" while
// disconnecting; that's fine — we treat that as gone).
deadline := time.Now().Add(teardownGracePeriod)
for time.Now().Before(deadline) {
    devicesOut, derr := a.executor.Execute(ctx,
    a.adbBinary(), "devices")
    if derr != nil {
        // Best effort; if adb itself fails, treat as kill-
    success
        // so we don't deadlock the matrix runner.
        return nil
    }
    stillAlive := false
    for _, line := range strings.Split(string(devicesOut),
    "\n") {
        if !strings.HasPrefix(line, target) {
            continue
        }
        fields := strings.Fields(line)
        if len(fields) >= 2 && fields[1] == "device" {
            stillAlive = true
            break
        }
    }
    if !stillAlive {
        return nil
    }
    select {
    case <-ctx.Done():
        return ctx.Err()
    case <-time.After(1 * time.Second):
    }
}

// Group B fast-path: the adb-emu-kill grace expired but the
// emulator is still in /proc. Try a port-strict force-kill
    via
// emulator.KillByPort. Matched==0 means no /proc entry passed
// the strict adjacent-token check — concurrent emulators on
// other ports are untouched, and we surface the original
// "did not exit" error so the matrix runner records an honest
// row failure.
report, kerr := killByPortHook(ctx, port)
if kerr != nil {
```

```
        // Forensic-only: log the KillByPort error but fall
        through
        // to the "did not exit" return. KillByPort errors are
        // best-effort signals, not gating ones.
        fmt.Fprintf(os.Stderr,
            "[teardown] KillByPort fast-path failed for port %d:
        %v\n",
            port, kerr,
        )
    }
    if report.Matched == 0 {
        return fmt.Errorf(
            "emulator on %s did not exit within %s after `adb emu
        kill`; KillByPort matched 0 processes (skip-on-mismatch
        safety)",
            target, teardownGracePeriod,
        )
    }
    // Re-poll briefly for /proc clearing.
    postDeadline := time.Now().Add(5 * time.Second)
    for time.Now().Before(postDeadline) {
        devicesOut, derr := a.executor.Execute(ctx,
        a.adbBinary(), "devices")
        if derr != nil {
            return nil
        }
        stillAlive := false
        for _, line := range strings.Split(string(devicesOut),
        "\n") {
            if !strings.HasPrefix(line, target) {
                continue
            }
            fields := strings.Fields(line)
            if len(fields) >= 2 && fields[1] == "device" {
                stillAlive = true
                break
            }
        }
        if !stillAlive {
            return nil
        }
        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-time.After(500 * time.Millisecond):
        }
    }
    return fmt.Errorf(

        "emulator on %s did not exit within %s + KillByPort grace;
        %d process(es) still alive (sigtermed=%v sigkilled=%v
        surviving=%v)",
        target, teardownGracePeriod,
```

```
        report.Matched, report.Sigtermed, report.Sigkilled,
        report.Surviving,
    )
}
```

- [ ] **Step 3: Run Teardown fast-path tests, confirm both pass**

```
cd submodules/containers
go test ./pkg/emulator/... -run 'TestTeardown_FastPath' -count=1
        -v
```

Expected: `--- PASS: TestTeardown_FastPath_SkipsOnMismatch`, `--- PASS: TestTeardown_FastPath_SucceedsAfterKillByPort`. PASS overall.

- [ ] **Step 4: Run the full emulator package test suite — confirm no regressions**

```
cd submodules/containers
go test ./pkg/emulator/... -count=1 -race
```

Expected: every test passes; no race detected.

- [ ] **Step 5: Falsifiability rehearsal — drop the skip-on-mismatch safety**

Temporarily change `if report.Matched == 0 { return fmt.Errorf(...) }` to `if false { ... }` (i.e., always treat KillByPort as success even when Matched==0).

Run:

```
cd submodules/containers
go test ./pkg/emulator/... -run
        TestTeardown_FastPath_SkipsOnMismatch -count=1 -v
```

Expected: `FAIL: TestTeardown_FastPath_SkipsOnMismatch` — the test asserts a non-nil error when KillByPort.Matched==0; the mutation makes Teardown return nil instead.

Save the observed failure verbatim for the commit body Bluff-Audit stamp.

- [ ] **Step 6: Revert the mutation, confirm tests pass again**

Restore `if report.Matched == 0 { return fmt.Errorf(...) }`. Run:

```
cd submodules/containers
go test ./pkg/emulator/... -run TestTeardown_FastPath -count=1 -v
```

Expected: PASS for both fast-path tests.

Task A5: types.go — DiagnosticInfo, FailureSummary, AVDRow ext, MatrixResult.Gating

Files:

- Modify: `submodules/containers/pkg/emulator/types.go`

☐ **Step 1: Add `DiagnosticInfo` and `FailureSummary` structs**

Append to `submodules/containers/pkg/emulator/types.go`:

```
// DiagnosticInfo is the per-AVD forensic snapshot captured
        immediately
// before instrumentation invocation. Reviewer-facing — answers
        the
// question "is the AVD the matrix claims it ran the AVD that
        actually
// ran?". A divergence between Diag.SDK and AVD.APILevel is the
// canonical "AVD shadow" bluff that scripts/tag.sh's Gate 3
        catches.
//
// Per clause 6.I clause 4: the AVD-row attestation MUST carry
        enough
// detail to verify the AVD identity post-hoc. Diag is that
        detail.
type DiagnosticInfo struct {
    // Target is the system-images package id the AVD was created
    // against, e.g. "system-
        images;android-34;google_apis;x86_64".
    // Empty when avdmanager is unavailable.
    Target string `json:"target,omitempty"`
    // SDK is ro.build.version.sdk reported by the booted emulator
    // (NOT the API level the AVD was created with — those usually
    // match but a misconfigured AVD can have a divergent runtime
        sdk).
    SDK int `json:"sdk,omitempty"`
    // Device is ro.product.model (preferred) or
        ro.product.device.
    Device string `json:"device,omitempty"`
    // ADBDevicesState is the line from `adb devices -l` for this
    // emulator's serial — the full record including
        "transport_id",
    // "product:", "model:", "device:" annotations.
    ADBDevicesState string `json:"adb_devices_state,omitempty"`
}
```

```
// FailureSummary is one parsed JUnit <failure> or <error> entry.
// Empty slice on TestResult.Passed=true. A synthetic entry with
// Type="<unparseable>" appears when the JUnit XML is missing or
// malformed — that is evidence corruption, NOT a row failure (the
// gating signal stays on TestResult.Passed per Sixth Law clause
        3).
type FailureSummary struct {
    Class   string `json:"class,omitempty"`
    Name    string `json:"name,omitempty"`
    Type    string `json:"type"` // "failure" | "error" |
        "<unparseable>"
    Message string `json:"message,omitempty"`
    Body    string `json:"body,omitempty"`
}
```

Step 2: Extend `TestResult` with `Diag`, `FailureSummaries`, `Concurrent`

In `submodules/containers/pkg/emulator/types.go`, find the `TestResult` struct (around lines 62-73) and replace with:

```
// TestResult captures the outcome of a single instrumentation-
        test
// execution against a booted emulator. Failed indicates whether
        the
// gradle task reported a non-zero exit AND/OR the JUnit XML
        reported
// any test class as failed.
type TestResult struct {
    AVD       AVD
    TestClass string
    Started   time.Time
    Duration  time.Duration
    Passed    bool
    // Output is the captured stdout+stderr of the test runner.
        May be
    // truncated for the matrix-report file; the full output stays
        in
    // the per-AVD log directory under EvidenceDir.
    Output string
    Error  error
    // Diag is the per-AVD forensic snapshot captured immediately
    // before instrumentation invocation. Group B clause 6.I
        extension.
    Diag DiagnosticInfo
    // FailureSummaries is the parsed JUnit XML <failure>/<error>
    // list for this AVD's run. Empty slice on Passed=true. Group
        B.
    FailureSummaries []FailureSummary
    // Concurrent is the matrix runner's --concurrent setting at
        the
    // time this test ran. 1 = serial (gating-eligible). Group B.
```

```
    Concurrent int
}
```

- [ ] **Step 3: Extend `MatrixResult` with `Gating`**

Find the `MatrixResult` struct (around lines 113-124) and replace with:

```
// MatrixResult holds the per-AVD outcomes from a single RunMatrix
        call.
type MatrixResult struct {
    Config     MatrixConfig
    Boots      []BootResult
    Tests      []TestResult
    StartedAt  time.Time
    FinishedAt time.Time
    // AttestationFile is the on-disk path the matrix runner wrote
        a
    // machine-readable attestation file to (typically
    // EvidenceDir/real-device-verification.json). Empty if the
        run
    // errored before the file could be written.
    AttestationFile string
    // Gating is true iff this matrix run is eligible to gate a
    // release tag. False when --concurrent != 1 OR --dev was set.
    // Group B clause 6.I extension; scripts/tag.sh refuses to
    // operate on attestations whose run-level Gating is false.
    Gating bool
}
```

- [ ] **Step 4: Extend `MatrixConfig` with `Concurrent` and `Dev`**

Find the `MatrixConfig` struct (around lines 78-110) and append two fields just before the closing `}`:

```
    // Concurrent is the maximum number of emulators booted in
        parallel
    // by RunMatrix. 1 = serial (gating-eligible; preserves all
    // existing behaviour). >1 = worker pool; sets
        MatrixResult.Gating
    // to false. Group B.
    Concurrent int

    // Dev marks the run as developer-iteration mode. Permits
        snapshot
    // reload (caller's choice) and sets MatrixResult.Gating to
        false.
    // Group B.
    Dev bool
```

- [ ] **Step 5: Confirm the package still compiles**

```
cd submodules/containers
go build ./pkg/emulator/...
```

Expected: build succeeds (no test changes needed yet — the new fields default to zero values, existing tests don't read them).

```
cd submodules/containers
go test ./pkg/emulator/... -count=1 -race
```

Expected: every test passes; no race detected.

Task A6: matrix.go — diag capture + JUnit XML parse + worker pool

Files:

- Modify: `submodules/containers/pkg/emulator/matrix.go`

☐ **Step 1: Add JUnit XML parser at the bottom of matrix.go**

Append to `submodules/containers/pkg/emulator/matrix.go`:

```
// parseJUnitFailures reads a single JUnit XML report file and
        returns
// every <failure> and <error> child of every <testcase> as a
// FailureSummary. Tolerates:
//   - Multiple <testsuite> siblings (Gradle's per-class XML
        output).
//   - <testcase> with both <failure> and <error> children — both
        are
//     captured as separate entries.
//   - <failure> / <error> with no message attribute (empty
        Message).
//   - <failure> / <error> with no text content (empty Body).
//
// Malformed XML returns a single synthetic FailureSummary with
// Type="<unparseable>". The synthetic entry is evidence
        corruption,
// NOT a row failure — the gating signal stays on
        TestResult.Passed
// per Sixth Law clause 3.
func parseJUnitFailures(xmlPath string) []FailureSummary {
    data, err := os.ReadFile(xmlPath)
    if err != nil {
        return []FailureSummary{{
            Type:    "<unparseable>",
            Message: fmt.Sprintf("junit-xml read failed: %v",
        err),
        }}
    }
```

```
type junitFailure struct {
    Message string `xml:"message,attr"`
    Type    string `xml:"type,attr"`
    Body    string `xml:",chardata"`
}
type junitTestcase struct {
    Class    string         `xml:"classname,attr"`
    Name     string         `xml:"name,attr"`
    Failures []junitFailure `xml:"failure"`
    Errors   []junitFailure `xml:"error"`
}
type junitTestsuite struct {
    Testcases []junitTestcase `xml:"testcase"`
}
type junitTestsuites struct {
    Suites []junitTestsuite `xml:"testsuite"`
}
// Decode into either <testsuites> or a single <testsuite>.
var suites []junitTestsuite
var ts junitTestsuites
if err := xml.Unmarshal(data, &ts); err == nil &&
    len(ts.Suites) > 0 {
    suites = ts.Suites
} else {
    var single junitTestsuite
    if err := xml.Unmarshal(data, &single); err != nil {
        return []FailureSummary{{
            Type:    "<unparseable>",
            Message: fmt.Sprintf("junit-xml parse failed:
    %v", err),
        }}
    }
    suites = []junitTestsuite{single}
}
var out []FailureSummary
for _, suite := range suites {
    for _, tc := range suite.Testcases {
        for _, f := range tc.Failures {
            out = append(out, FailureSummary{
                Class:   tc.Class,
                Name:    tc.Name,
                Type:    "failure",
                Message: f.Message,
                Body:    f.Body,
            })
        }
        for _, e := range tc.Errors {
            out = append(out, FailureSummary{
                Class:   tc.Class,
                Name:    tc.Name,
                Type:    "error",
                Message: e.Message,
                Body:    e.Body,
```

```
                })
            }
        }
    }
    if out == nil {
        return []FailureSummary{}
    }
    return out
}
```

- [ ] **Step 2: Add `encoding/xml` to the import block**

In `submodules/containers/pkg/emulator/matrix.go`, change the import block (lines 3-10) from:

```
import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "time"
)
```

to:

```
import (
    "context"
    "encoding/json"
    "encoding/xml"
    "fmt"
    "os"
    "path/filepath"
    "sync"
    "time"
)
```

(`sync` is added in advance for the worker pool in step 4.)

- [ ] **Step 3: Wire diag capture + JUnit parsing into `RunMatrix` (single-AVD inner block)**

Extract the per-AVD work from `RunMatrix` into a helper `runOne` that captures diag + parses JUnit XML. In `submodules/containers/pkg/emulator/matrix.go`, after the `defaultIfZero` helper (around line 34), add:

```
// runOne executes one (boot → install → test → teardown) cycle
        for a
// single AVD. Returns the BootResult and TestResult ready to
        append to
```

```
// MatrixResult.Boots / .Tests. Captures diag pre-test and parses
        JUnit
// XML post-test per Group B.
//
// runOne is invoked sequentially in serial mode and concurrently
        from
// a worker pool when MatrixConfig.Concurrent > 1. Each invocation
        owns
// its own emulator instance, so concurrent calls are safe as long
        as
// the underlying Emulator implementation is too (AndroidEmulator
// satisfies that — Boot()'s discovery picks an unused console
        port).
func (r *AndroidMatrixRunner) runOne(
    ctx context.Context,
    avd AVD,
    config MatrixConfig,
    bootTimeout time.Duration,
    testTimeout time.Duration,
) (BootResult, TestResult) {
    boot, err := r.emulator.Boot(ctx, avd, config.ColdBoot)
    if err != nil {
        return boot, TestResult{
            AVD:        avd,
            TestClass:  config.TestClass,
            Started:    time.Now(),
            Passed:     false,
            Error:      fmt.Errorf("boot failed: %w", err),
            Concurrent: maxInt(config.Concurrent, 1),
        }
    }
    waitDuration, err := r.emulator.WaitForBoot(ctx,
        boot.ADBPort, bootTimeout)
    boot.BootDuration += waitDuration
    if err != nil {
        boot.Error = err
        _ = r.emulator.Teardown(ctx, boot.ADBPort)
        return boot, TestResult{
            AVD:        avd,
            TestClass:  config.TestClass,
            Started:    time.Now(),
            Passed:     false,
            Error:      fmt.Errorf("wait-for-boot failed: %w",
        err),
            Concurrent: maxInt(config.Concurrent, 1),
        }
    }
    boot.BootCompleted = true

    if err := r.emulator.Install(ctx, boot.ADBPort,
        config.APKPath); err != nil {
        _ = r.emulator.Teardown(ctx, boot.ADBPort)
        return boot, TestResult{
            AVD:        avd,
```

```
        TestClass:  config.TestClass,
        Started:    time.Now(),
        Passed:     false,
        Error:      fmt.Errorf("install failed: %w", err),
        Concurrent: maxInt(config.Concurrent, 1),
    }
}

// Diagnostic capture happens here, between Install and the
    test —
// after Android is up and the APK is installed (so the
    emulator is
// stable) and before the test runs (so the diag reflects the
// state the test will encounter).
diag := r.captureDiagnostic(ctx, boot.ADBPort, avd)

startedTest := time.Now()
out, passed, runErr := r.emulator.RunInstrumentation(
    ctx, boot.ADBPort, config.TestClass, testTimeout,
)
test := TestResult{
    AVD:        avd,
    TestClass:  config.TestClass,
    Started:    startedTest,
    Duration:   time.Since(startedTest),
    Passed:     passed,
    Output:     out,
    Error:      runErr,
    Diag:       diag,
    Concurrent: maxInt(config.Concurrent, 1),
}

// Persist per-AVD evidence (gradle.log + JUnit XML test-
    report).
avdDir := filepath.Join(config.EvidenceDir, avd.Name)
if mkErr := os.MkdirAll(avdDir, 0o755); mkErr == nil {
    logPath := filepath.Join(avdDir, "gradle.log")
    if werr := os.WriteFile(logPath, []byte(out), 0o644);
    werr != nil {
        fmt.Fprintf(os.Stderr,
            "[matrix] warning: failed to persist gradle.log
    for %s: %v\n",
            avd.Name, werr,
        )
    }
    matches, _ := filepath.Glob("app/build/outputs/
    androidTest-results/connected/debug/TEST-*.xml")
    if len(matches) > 0 {
        reportDir := filepath.Join(avdDir, "test-report")
        _ = os.MkdirAll(reportDir, 0o755)
        for _, src := range matches {
            data, rerr := os.ReadFile(src)
            if rerr != nil {
                continue
```

```
                }
                dst := filepath.Join(reportDir,
        filepath.Base(src))
                _ = os.WriteFile(dst, data, 0o644)
            }
            // Parse JUnit failures from every copied XML;
        aggregate
            // across files (Gradle emits one XML per class).
            var summaries []FailureSummary
            reportEntries, _ := os.ReadDir(reportDir)
            for _, ent := range reportEntries {
                if ent.IsDir() || !strings.HasSuffix(ent.Name(),
        ".xml") {
                    continue
                }
                summaries = append(summaries,
        parseJUnitFailures(filepath.Join(reportDir,
        ent.Name()))...)
            }
            test.FailureSummaries = summaries
        }
    }
    if test.FailureSummaries == nil {
        test.FailureSummaries = []FailureSummary{}
    }
    _ = r.emulator.Teardown(ctx, boot.ADBPort)
    return boot, test
}

// captureDiagnostic gathers the per-AVD forensic snapshot used by
        Group
// B's clause 6.I extension. Best-effort: missing fields default
        to zero
// values so a partial diag is recorded rather than no diag at
        all.
func (r *AndroidMatrixRunner) captureDiagnostic(ctx
        context.Context, port int, avd AVD) DiagnosticInfo {
    d := DiagnosticInfo{}
    if ae, ok := r.emulator.(*AndroidEmulator); ok {
        target := fmt.Sprintf("localhost:%d", port)
        if sdkOut, err := ae.executor.Execute(ctx,
        ae.adbBinary(), "-s", target, "shell", "getprop",
        "ro.build.version.sdk"); err == nil {
            if sdk, perr :=
        strconv.Atoi(strings.TrimSpace(string(sdkOut))); perr ==
        nil {
                d.SDK = sdk
            }
        }
        if modelOut, err := ae.executor.Execute(ctx,
        ae.adbBinary(), "-s", target, "shell", "getprop",
        "ro.product.model"); err == nil &&
        strings.TrimSpace(string(modelOut)) != "" {
            d.Device = strings.TrimSpace(string(modelOut))
```

```
        } else if devOut, err := ae.executor.Execute(ctx,
        ae.adbBinary(), "-s", target, "shell", "getprop",
        "ro.product.device"); err == nil {
            d.Device = strings.TrimSpace(string(devOut))
        }
        if devicesOut, err := ae.executor.Execute(ctx,
        ae.adbBinary(), "devices", "-l"); err == nil {
            for _, line := range
        strings.Split(string(devicesOut), "\n") {
                if strings.HasPrefix(line, target) {
                    d.ADBDevicesState = strings.TrimSpace(line)
                    break
                }
            }
        }
        // Target (system-images package id) — best-effort via
        // avdmanager. The CLI is `avdmanager list avd -c` for
        compact
        // output; falling back to empty is acceptable.
        avdmanager := filepath.Join(ae.androidSdkRoot, "cmdline-
        tools", "latest", "bin", "avdmanager")
        if avdmanagerOut, err := ae.executor.Execute(ctx,
        avdmanager, "list", "avd"); err == nil {
            d.Target = parseAVDTarget(string(avdmanagerOut),
        avd.Name)
        }
    }
    return d
}

// parseAVDTarget walks `avdmanager list avd` text output and
        finds the
// "Based on:" line for the AVD with the requested name. Returns
        empty
// string when not found.
func parseAVDTarget(out string, avdName string) string {
    lines := strings.Split(out, "\n")
    for i, line := range lines {
        nameField := strings.TrimSpace(line)
        if !strings.HasPrefix(nameField, "Name:") {
            continue
        }
        got := strings.TrimSpace(strings.TrimPrefix(nameField,
        "Name:"))
        if got != avdName {
            continue
        }
        // Look for "Based on: <package id>" within the next 10
        lines
        for j := i + 1; j < len(lines) && j < i+10; j++ {
            tl := strings.TrimSpace(lines[j])
            if strings.HasPrefix(tl, "Based on:") {
                return strings.TrimSpace(strings.TrimPrefix(tl,
        "Based on:"))
```

```
            }
        }
    }
    return ""
}

func maxInt(a, b int) int {
    if a > b {
        return a
    }
    return b
}
```

☐ **Step 4: Add `strconv` and `strings` to the import block**

The new helper functions require `strconv` and `strings`. Update the import block to:

```
import (
    "context"
    "encoding/json"
    "encoding/xml"
    "fmt"
    "os"
    "path/filepath"
    "strconv"
    "strings"
    "sync"
    "time"
)
```

☐ **Step 5: Refactor `RunMatrix` to call `runOne` (serial) or dispatch to a worker pool (concurrent)**

Replace the `RunMatrix` function body (lines 44-179 of the original) with the version below. Keep the validation block (lines 48-62) intact; replace ONLY the per-AVD loop and the assignment of `result.Tests` / `result.Boots`:

```
func (r *AndroidMatrixRunner) RunMatrix(
    ctx context.Context,
    config MatrixConfig,
) (MatrixResult, error) {
    if len(config.AVDs) == 0 {
        return MatrixResult{}, fmt.Errorf("MatrixConfig.AVDs is
        empty")
    }
    if config.APKPath == "" {
        return MatrixResult{},
        fmt.Errorf("MatrixConfig.APKPath is empty")
    }
```

```
if config.TestClass == "" {
    return MatrixResult{}, fmt.Errorf("MatrixConfig.TestClass
    is empty")
}
if config.EvidenceDir == "" {
    return MatrixResult{},
    fmt.Errorf("MatrixConfig.EvidenceDir is empty")
}
if err := os.MkdirAll(config.EvidenceDir, 0o755); err != nil {
    return MatrixResult{}, fmt.Errorf("create evidence dir:
    %w", err)
}

bootTimeout := defaultIfZero(config.BootTimeout,
    5*time.Minute)
testTimeout := defaultIfZero(config.TestTimeout,
    10*time.Minute)
concurrent := config.Concurrent
if concurrent < 1 {
    concurrent = 1
}

result := MatrixResult{
    Config:    config,
    StartedAt: time.Now(),
    Gating:    concurrent == 1 && !config.Dev,
}

if concurrent == 1 {
    // Serial path — preserves existing behaviour.
    for _, avd := range config.AVDs {
        boot, test := r.runOne(ctx, avd, config, bootTimeout,
    testTimeout)
        result.Boots = append(result.Boots, boot)
        result.Tests = append(result.Tests, test)
    }
} else {
    // Concurrent path — bounded worker pool. Each worker
    pulls
    // AVDs off a channel and runs runOne; results land on a
    // buffered channel that is drained after all workers
    exit.
    type pair struct {
        boot BootResult
        test TestResult
    }
    queue := make(chan AVD)
    results := make(chan pair, len(config.AVDs))
    var wg sync.WaitGroup
    for w := 0; w < concurrent; w++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for avd := range queue {
```

```
                boot, test := r.runOne(ctx, avd, config,
        bootTimeout, testTimeout)
                results <- pair{boot: boot, test: test}
            }
        }()
    }
    for _, avd := range config.AVDs {
        queue <- avd
    }
    close(queue)
    wg.Wait()
    close(results)
    for p := range results {
        result.Boots = append(result.Boots, p.boot)
        result.Tests = append(result.Tests, p.test)
    }
}

result.FinishedAt = time.Now()
attestationFile := filepath.Join(config.EvidenceDir, "real-
    device-verification.json")
if err := writeAttestation(attestationFile, result); err ==
    nil {
    result.AttestationFile = attestationFile
}
return result, nil
}
```

☐ **Step 6: Extend `writeAttestation` to emit the new fields (gating, diag, failure_summaries, concurrent)**

Replace the `writeAttestation` function (lines 181-232 of the original):

```
func writeAttestation(path string, r MatrixResult) error {
    type rowJSON struct {
        AVD             string          `json:"avd"`
        APILevel        int
        `json:"api_level,omitempty"`
        FormFactor      string
        `json:"form_factor,omitempty"`
        BootSeconds     float64         `json:"boot_seconds"`
        BootError       string
        `json:"boot_error,omitempty"`
        TestClass       string          `json:"test_class"`
        TestPassed      bool            `json:"test_passed"`
        TestSeconds     float64         `json:"test_seconds"`
        TestError       string
        `json:"test_error,omitempty"`
        GradleLogPath   string
        `json:"gradle_log_path,omitempty"`
        Diag            DiagnosticInfo  `json:"diag"`
        FailureSummaries []FailureSummary
        `json:"failure_summaries"`
```

```
    Concurrent       int              `json:"concurrent"`
}
rows := make([]rowJSON, 0, len(r.Tests))
for i, t := range r.Tests {
    var bootSec float64
    var bootErr string
    if i < len(r.Boots) {
        bootSec = r.Boots[i].BootDuration.Seconds()
        if r.Boots[i].Error != nil {
            bootErr = r.Boots[i].Error.Error()
        }
    }
    var testErr string
    if t.Error != nil {
        testErr = t.Error.Error()
    }
    summaries := t.FailureSummaries
    if summaries == nil {
        summaries = []FailureSummary{}
    }
    concurrent := t.Concurrent
    if concurrent < 1 {
        concurrent = 1
    }
    rows = append(rows, rowJSON{
        AVD:              t.AVD.Name,
        APILevel:         t.AVD.APILevel,
        FormFactor:       t.AVD.FormFactor,
        BootSeconds:      bootSec,
        BootError:        bootErr,
        TestClass:        t.TestClass,
        TestPassed:       t.Passed,
        TestSeconds:      t.Duration.Seconds(),
        TestError:        testErr,
        GradleLogPath:    filepath.Join(t.AVD.Name,
    "gradle.log"),
        Diag:             t.Diag,
        FailureSummaries: summaries,
        Concurrent:       concurrent,
    })
}
doc := map[string]any{
    "started_at":  r.StartedAt.Format(time.RFC3339),
    "finished_at": r.FinishedAt.Format(time.RFC3339),
    "all_passed":  r.AllPassed(),
    "gating":      r.Gating,
    "rows":        rows,
}
bytes, err := json.MarshalIndent(doc, "", "  ")
if err != nil {
    return err
}
```

```
    return os.WriteFile(path, bytes, 0o644)
}
```

- [ ] **Step 7: Confirm package compiles + existing tests pass**

```
cd submodules/containers
go build ./pkg/emulator/...
go test ./pkg/emulator/... -count=1 -race
```

Expected: build succeeds; ALL existing tests pass (including the matrix tests added in Group A-prime — they don't read the new fields).

If `TestAndroidMatrixRunner_AllAVDsPass_ReportsAllPassed` or `TestAndroidMatrixRunner_GradleLogWriteFailure_DoesNotFailRun` fail, inspect the JSON the test reads — the new fields have JSON tags but should not break existing assertions.

Task A7: matrix.go tests — JUnit parser + Gating field

Files:

- Modify: `submodules/containers/pkg/emulator/matrix_test.go`

- [ ] **Step 1: Add JUnit parser tests**

Append to `submodules/containers/pkg/emulator/matrix_test.go`:

```
// ---------------------------------------------------------------------
// JUnit XML parser tests — Group B clause 6.I extension
//
// parseJUnitFailures MUST tolerate Gradle's per-class XML output (one
// <testsuite> per file, sometimes wrapped in <testsuites>), recover
// every <failure> AND <error> entry, and degrade gracefully on
// missing/malformed input by emitting a single synthetic <unparseable>
// entry that does NOT mark the row as failed (the gating signal stays
// on TestResult.Passed per Sixth Law clause 3).
// ---------------------------------------------------------------------

func TestParseJUnitFailures_AllPass_EmptySlice(t *testing.T) {
    dir := t.TempDir()
    path := filepath.Join(dir, "TEST-pass.xml")
    xmlBody := `<?xml version="1.0" encoding="UTF-8"?>
<testsuite name="lava.app.SomeTest" tests="2" failures="0" errors="0">
  <testcase classname="lava.app.SomeTest" name="testA"/>
```

```
  <testcase classname="lava.app.SomeTest" name="testB"/>
</testsuite>`
    if err := os.WriteFile(path, []byte(xmlBody), 0o644); err !=
        nil {
        t.Fatalf("write fixture: %v", err)
    }
    got := parseJUnitFailures(path)
    if len(got) != 0 {
        t.Fatalf("expected empty slice on all-pass, got %d
        entries: %+v", len(got), got)
    }
}

func TestParseJUnitFailures_FailureAndError_BothCaptured(t
        *testing.T) {
    dir := t.TempDir()
    path := filepath.Join(dir, "TEST-mixed.xml")
    xmlBody := `<?xml version="1.0" encoding="UTF-8"?>
<testsuite name="lava.app.SomeTest" tests="2" failures="1"
        errors="1">
  <testcase classname="lava.app.SomeTest" name="testFail">
    <failure message="expected 1 got 2"
        type="java.lang.AssertionError">stack trace lines here</
        failure>
  </testcase>
  <testcase classname="lava.app.SomeTest" name="testError">
    <error message="NPE" type="java.lang.NullPointerException">at
        lava.app.Foo.bar(Foo.kt:42)</error>
  </testcase>
</testsuite>`
    if err := os.WriteFile(path, []byte(xmlBody), 0o644); err !=
        nil {
        t.Fatalf("write fixture: %v", err)
    }
    got := parseJUnitFailures(path)
    if len(got) != 2 {
        t.Fatalf("expected 2 entries (1 failure + 1 error), got
        %d: %+v", len(got), got)
    }
    var seenFailure, seenError bool
    for _, fs := range got {
        if fs.Type == "failure" && fs.Name == "testFail" &&
        fs.Message == "expected 1 got 2" {
            seenFailure = true
        }
        if fs.Type == "error" && fs.Name == "testError" &&
        fs.Message == "NPE" {
            seenError = true
        }
    }
    if !seenFailure || !seenError {
        t.Fatalf("missing failure/error entries: seenFailure=%v
        seenError=%v all=%+v",
            seenFailure, seenError, got)
```

```
    }
}

func TestParseJUnitFailures_MultiTestsuites_Wrapper(t *testing.T)
        {
    dir := t.TempDir()
    path := filepath.Join(dir, "TEST-wrapped.xml")
    xmlBody := `<?xml version="1.0" encoding="UTF-8"?>
<testsuites>
  <testsuite name="lava.app.A" tests="1" failures="1">
    <testcase classname="lava.app.A" name="t1">
      <failure message="A failed">trace A</failure>
    </testcase>
  </testsuite>
  <testsuite name="lava.app.B" tests="1" failures="1">
    <testcase classname="lava.app.B" name="t2">
      <failure message="B failed">trace B</failure>
    </testcase>
  </testsuite>
</testsuites>`
    if err := os.WriteFile(path, []byte(xmlBody), 0o644); err !=
        nil {
        t.Fatalf("write fixture: %v", err)
    }
    got := parseJUnitFailures(path)
    if len(got) != 2 {
        t.Fatalf("expected 2 entries from 2 testsuites, got %d: %
        +v", len(got), got)
    }
}

func TestParseJUnitFailures_MalformedXML_SyntheticEntry(t
        *testing.T) {
    dir := t.TempDir()
    path := filepath.Join(dir, "TEST-broken.xml")
    if err := os.WriteFile(path, []byte("<testsuite><tes"),
        0o644); err != nil {
        t.Fatalf("write fixture: %v", err)
    }
    got := parseJUnitFailures(path)
    if len(got) != 1 {

        t.Fatalf("expected 1 synthetic entry on malformed XML, got
        %d: %+v", len(got), got)
    }
    if got[0].Type != "<unparseable>" {
        t.Fatalf("expected Type=<unparseable>, got %q",
        got[0].Type)
    }
}

func TestParseJUnitFailures_MissingFile_SyntheticEntry(t
        *testing.T) {
    got := parseJUnitFailures("/no/such/file.xml")
```

```
    if len(got) != 1 {
        t.Fatalf("expected 1 synthetic entry on missing file, got
        %d", len(got))
    }
    if got[0].Type != "<unparseable>" {
        t.Fatalf("expected Type=<unparseable>, got %q",
        got[0].Type)
    }
}
```

☐

Step 2: Add Gating-field tests using a fake Emulator

Append to `submodules/containers/pkg/emulator/matrix_test.go`:

```
// ---------------------------------------------------------------------
// Gating-flag tests — Group B
//
// MatrixResult.Gating is true ONLY when --concurrent == 1 AND --
        dev is
// false. Either flag flips it false; tag.sh refuses non-gating
// attestations. Defaults preserve existing behaviour
        (gating=true).
// ---------------------------------------------------------------------

// stubEmulator is a minimal Emulator that always succeeds and
        returns
// canned ports. Used to drive RunMatrix without hitting the real
        adb.
type stubEmulator struct {
    port int // monotonically incremented per Boot
}

func (s *stubEmulator) Boot(_ context.Context, avd AVD, _ bool)
        (BootResult, error) {
    s.port += 2
    return BootResult{
        AVD:         avd,
        Started:     true,
        ConsolePort: s.port,
        ADBPort:     s.port + 1,
    }, nil
}
func (s *stubEmulator) WaitForBoot(_ context.Context, _ int, _
        time.Duration) (time.Duration, error) {
    return 0, nil
}
func (s *stubEmulator) Install(_ context.Context, _ int, _
        string) error { return nil }
func (s *stubEmulator) RunInstrumentation(_ context.Context, _
        int, _ string, _ time.Duration) (string, bool, error) {
    return "BUILD SUCCESSFUL", true, nil
}
```

```
func (s *stubEmulator) Teardown(_ context.Context, _ int) error {
        return nil }

func runMatrixWithStub(t *testing.T, concurrent int, dev bool)
        MatrixResult {
    t.Helper()
    dir := t.TempDir()
    apkPath := filepath.Join(dir, "app-debug.apk")
    if err := os.WriteFile(apkPath, []byte("fake apk bytes"),
        0o644); err != nil {
        t.Fatalf("write fixture apk: %v", err)
    }
    r := NewAndroidMatrixRunner(&stubEmulator{})
    res, err := r.RunMatrix(context.Background(), MatrixConfig{
        AVDs:        []AVD{{Name: "A1", APILevel: 28}, {Name:
        "A2", APILevel: 30}},
        APKPath:     apkPath,
        TestClass:   "lava.app.X",
        EvidenceDir: dir,
        Concurrent:  concurrent,
        Dev:         dev,
    })
    if err != nil {
        t.Fatalf("RunMatrix returned error: %v", err)
    }
    return res
}

func TestRunMatrix_Gating_TrueOnDefaults(t *testing.T) {
    res := runMatrixWithStub(t, 0, false) // 0 → coerced to 1
        (serial)
    if !res.Gating {
        t.Fatalf("expected Gating=true on defaults (serial, non-
        dev), got false")
    }
}

func TestRunMatrix_Gating_FalseOnConcurrent(t *testing.T) {
    res := runMatrixWithStub(t, 2, false)
    if res.Gating {
        t.Fatalf("expected Gating=false when Concurrent=2, got
        true")
    }
}

func TestRunMatrix_Gating_FalseOnDev(t *testing.T) {
    res := runMatrixWithStub(t, 1, true)
    if res.Gating {
        t.Fatalf("expected Gating=false when Dev=true, got true")
    }
}
```

Step 3: Run the new tests, confirm all 8 pass

```
cd submodules/containers
go test ./pkg/emulator/... -run 'TestParseJUnitFailures|
        TestRunMatrix_Gating' -count=1 -v
```

Expected: PASS for all 5 parser tests + 3 gating tests.

- [ ]

Step 4: Run the full emulator package test suite

```
cd submodules/containers
go test ./pkg/emulator/... -count=1 -race
```

Expected: every test passes; no race detected.

- [ ]

Step 5: Falsifiability rehearsal — drop `<error>` element handling

In `parseJUnitFailures` (matrix.go), comment out the entire `for _, e := range tc.Errors {...}` block.

Run:

```
cd submodules/containers
go test ./pkg/emulator/... -run
        TestParseJUnitFailures_FailureAndError_BothCaptured
        -count=1 -v
```

Expected: `FAIL — expected 2 entries (1 failure + 1 error), got 1: [{Class:lava.app.SomeTest Name:testFail Type:failure Message:expected 1 got 2 ...}]`.

Save the observed failure verbatim for the commit body Bluff-Audit stamp.

Restore the `<error>` loop. Run again — PASS.

- [ ]

Step 6: Falsifiability rehearsal — flip Gating default to false

In `RunMatrix`, change `Gating: concurrent == 1 && !config.Dev` to `Gating: false`.

Run:

```
cd submodules/containers
go test ./pkg/emulator/... -run
        TestRunMatrix_Gating_TrueOnDefaults -count=1 -v
```

Expected: `FAIL: TestRunMatrix_Gating_TrueOnDefaults` — assertion message says `expected Gating=true on defaults (serial, non-dev), got false`.

Save the observed failure verbatim. Restore the original line. Re-run, confirm PASS.

Task A8: cmd/emulator-matrix — --concurrent + --dev flags

Files:

- Modify: `submodules/containers/cmd/emulator-matrix/main.go`

☐

Step 1: Add the two new flag definitions inside `main()`

In `submodules/containers/cmd/emulator-matrix/main.go`, insert after the existing `flagTestTimeout` (around line 91):

```
flagConcurrent := flag.Int("concurrent", 1,
    "Max concurrent emulators (default 1; values >1 set
    MatrixResult.Gating=false)")
flagDev := flag.Bool("dev", false,
    "Developer-iteration mode; permits snapshot reload, sets
    MatrixResult.Gating=false")
```

☐

Step 2: Pipe the flags into `MatrixConfig`

Find the `runner.RunMatrix(ctx, emulator.MatrixConfig{...})` call (around line 120). Add the two new fields:

```
result, err := runner.RunMatrix(ctx, emulator.MatrixConfig{
    AVDs:           avds,
    AndroidSdkRoot: *flagSdkRoot,
    APKPath:        *flagAPK,
    TestClass:      *flagTestClass,
    EvidenceDir:    *flagEvidence,
    BootTimeout:    *flagBootTimeout,
    TestTimeout:    *flagTestTimeout,
    ColdBoot:       *flagColdBoot,
    Concurrent:     *flagConcurrent,
    Dev:            *flagDev,
})
```

☐

Step 3: Add a one-line summary print after the existing summary (so the operator sees gating status without inspecting JSON)

Find the loop printing per-row results (around lines 134-145). After the `if !result.AllPassed()` block (around line 146), insert before that block:

```
    if result.Gating {
        fmt.Println("Gating: TRUE  (serial run, --dev=false —
        clause-6.I-clause-7-eligible)")
    } else {
        fmt.Println("Gating: FALSE (--concurrent>1 OR --dev —
        tag.sh will refuse this attestation)")
    }
```

- [ ] **Step 4: Confirm the binary builds**

```
cd submodules/containers
go build ./cmd/emulator-matrix/...
```

Expected: build succeeds.

- [ ] **Step 5: Confirm the help output advertises the new flags**

```
cd submodules/containers
go run ./cmd/emulator-matrix/ --help 2>&1 | head -40
```

Expected: output contains `--concurrent int` and `--dev` lines with the descriptions from step 1.

Task A9: Phase A commit + falsifiability stamps

Files:

- (no edits, just commit)

- [ ] **Step 1: Stage every Phase A change**

```
cd submodules/containers
git status
git add pkg/emulator/cleanup.go pkg/emulator/cleanup_test.go \
        pkg/emulator/types.go \
        pkg/emulator/android.go pkg/emulator/android_test.go \
        pkg/emulator/matrix.go pkg/emulator/matrix_test.go \
        cmd/emulator-matrix/main.go
git status
```

Expected: all 8 files staged for commit; nothing else unstaged.

- [ ] **Step 2: Run the full test suite one final time**

```
cd submodules/containers
go test ./pkg/emulator/... -count=1 -race
go build ./pkg/emulator/... ./cmd/emulator-matrix/...
```

Expected: every test passes; both builds succeed.

- [ ] **Step 3: Commit with the 5 Bluff-Audit stamps**

The commit body MUST contain 5 mutation rehearsals (KillByPort substring-safety + Teardown skip-on-mismatch + JUnit-parser drop-error-elements + Gating-default-true + one for the worker pool). Use the captured failure messages from earlier steps verbatim. Use a HEREDOC for formatting:

```
cd submodules/containers
git commit -m "$(cat <<'EOF'
feat(emulator): Group B — KillByPort fast-path + per-row diag +
        JUnit parsing + concurrent mode

Five matrix-runner tightenings, all bound to the parent Lava
        project's
Group B spec at docs/superpowers/specs/2026-05-05-anti-bluff-
        mandate-reinforcement-group-b-design.md
(commit e2a3af2). None relax existing clauses; all close the next
class of bluffs the gate cannot yet detect.

1. cleanup.go:KillByPort(ctx, port) — strict adjacent-token /proc
        walk
   targeting the QEMU instance whose argv contains "-port <port>"
   adjacent. Concurrent emulators on other ports / sibling-project
   QEMUs / developer-spawned QEMUs are NEVER touched. Matched=0 is
        a
   no-op safe state.
2. android.go:Teardown — after the existing 30s adb-emu-kill
        grace,
   invoke killByPortHook(port). On Matched=0, return the original
   "did not exit" error (skip-on-mismatch safety). On match, re-
        poll
   for /proc clearing.
3. types.go — DiagnosticInfo (target/sdk/device/
        adb_devices_state),
   FailureSummary, KillReport. TestResult.Diag + .FailureSummaries
        +
   .Concurrent. MatrixResult.Gating. MatrixConfig.Concurrent
        + .Dev.
4. matrix.go — runOne extracted; captureDiagnostic between Install
   and RunInstrumentation; parseJUnitFailures post-test populates
   FailureSummaries; worker pool when Concurrent>1;
        writeAttestation
   emits the new fields.
5. cmd/emulator-matrix — --concurrent N (default 1) + --dev flags;
   either flips MatrixResult.Gating to false. Operator-facing
   summary line announces gating status.
```

```
Bluff-Audit (5 mutation rehearsals, all reverted):

  Test:     TestKillByPort_SubstringSafety
  Mutation: weaken matcher to strings.Contains(token, target)
  Observed: expected Matched=0 (no adjacent token pair), got 2
  Reverted: yes

  Test:     TestTeardown_FastPath_SkipsOnMismatch
  Mutation: drop the `if report.Matched == 0` early-return guard
  Observed: expected Teardown to return an error when
        KillByPort.Matched==0
            and emulator persists, got nil
  Reverted: yes

  Test:     TestParseJUnitFailures_FailureAndError_BothCaptured
  Mutation: comment out the for _, e := range tc.Errors loop
  Observed: expected 2 entries (1 failure + 1 error), got 1
  Reverted: yes

  Test:     TestRunMatrix_Gating_TrueOnDefaults
  Mutation: change `Gating: concurrent == 1 && !config.Dev` to
        `Gating: false`
  Observed: expected Gating=true on defaults (serial, non-dev),
        got false
  Reverted: yes

  Test:     TestRunMatrix_Gating_FalseOnConcurrent
  Mutation: change `Gating: concurrent == 1 && !config.Dev` to
        `Gating: true`
  Observed: expected Gating=false when Concurrent=2, got true
  Reverted: yes

Runtime evidence:

  go test ./pkg/emulator/... -count=1 -race  → PASS
  go build ./pkg/emulator/... ./cmd/emulator-matrix/...  → ok
  go run ./cmd/emulator-matrix/ --help  → --concurrent int / --dev
        advertised

Constitutional bindings: parent §6.I clause 4 + clause 7 (matrix-
        row
schema extension), §6.J / §6.L (anti-bluff functional-reality
mandate), §6.N.1 (per-matrix-runner change → Bluff-Audit stamp
mandatory), §6.N.2 (production-code coverage of gate-shaping
        files).

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Step 4: Push the new branch to all 4 Containers upstreams

```
cd submodules/containers
for r in github gitlab gitflic gitverse; do
  echo "=== $r ==="
  git push -u "$r" lava-pin/2026-05-06-group-b
done
```

Expected: each push reports `* [new branch] lava-pin/2026-05-06-group-b -> lava-pin/2026-05-06-group-b`.

If any push errors, STOP — diagnose the upstream issue and re-push only the failing remote.

☐ **Step 5: Verify 4-mirror SHA convergence via live ls-remote**

```
cd submodules/containers
for r in github gitlab gitflic gitverse; do
  sha=$(git ls-remote "$r" refs/heads/lava-pin/2026-05-06-group-b
        2>/dev/null | awk '{print $1}')
  echo "$r: $sha"
done
```

Expected: identical 40-char SHA from all 4 remotes.

If any remote shows a different SHA, investigate (network, permissions). Do NOT proceed to Phase B until convergence.

Phase B — Lava parent code (1 commit on `master`)

> Working tree: `/run/media/milosvasic/DATA4TB/Projects/Lava` (the Lava parent). Tests run via `bash tests/tag-helper/run_all.sh` plus the existing `bash tests/pre-push/check{4,5}_test.sh` (already passing from Group A-prime).

Task B1: scripts/run-emulator-tests.sh — auto-detect + --tag + passthrough flags

Files:

- Modify: `scripts/run-emulator-tests.sh`

☐ **Step 1: Add the new flag declarations + auto-detect helper**

Edit `scripts/run-emulator-tests.sh`. Find the defaults block (currently around lines 44-53):

```
DEFAULT_TEST_CLASS="lava.app.challenges.Challenge01AppLaunchAndTrackerSelectionTes
DEFAULT_AVDS="CZ_API28_Phone:28:phone,CZ_API30_Phone:30:phone,CZ_API34_Phone:34:ph
DEFAULT_EVIDENCE_DIR=".lava-ci-evidence/$(date -u +%Y-%m-%dT%H-%M-
        %SZ)-matrix"

TEST_CLASS="$DEFAULT_TEST_CLASS"
AVDS="$DEFAULT_AVDS"
EVIDENCE_DIR="$DEFAULT_EVIDENCE_DIR"
BUILD_APK=1
BOOT_TIMEOUT=""
```

Replace with:

```
DEFAULT_TEST_CLASS="lava.app.challenges.Challenge01AppLaunchAndTrackerSelectionTes
DEFAULT_AVDS="CZ_API28_Phone:28:phone,CZ_API30_Phone:30:phone,CZ_API34_Phone:34:ph

TEST_CLASS="$DEFAULT_TEST_CLASS"
AVDS="$DEFAULT_AVDS"
EVIDENCE_DIR=""           # set by detect_evidence_dir below
TAG_OVERRIDE=""
BUILD_APK=1
BOOT_TIMEOUT=""
CONCURRENT=""
DEV_MODE=0

# detect_version_prefix returns "Lava-Android-<versionName>-
        <versionCode>"
# parsed from app/build.gradle.kts. Echoes "Lava-Android-unknown"
        when
# the file is missing or unparseable so an iteration run still
        proceeds
# (tag.sh's gates will reject the resulting evidence anyway).
detect_version_prefix() {
    local f="$PROJECT_DIR/app/build.gradle.kts"
    if [[ ! -f "$f" ]]; then
        echo "Lava-Android-unknown"
        return 0
    fi
    local v vc
    v=$(grep -oE 'versionName[[:space:]]+"[^"]+"' "$f" | head -n1
        | sed -E 's/.*"([^"]+)".*/\1/')
    vc=$(grep -oE 'versionCode[[:space:]]+[0-9]+' "$f" | head -n1
        | awk '{print $NF}')
    if [[ -z "$v" || -z "$vc" ]]; then
        echo "Lava-Android-unknown"
        return 0
    fi
    echo "Lava-Android-${v}-${vc}"
}
```

☐

Step 2: Extend the option parser

Find the option-parsing `while` loop (currently around lines 55-81). Replace its body to accept `--tag`, `--concurrent`, `--dev`:

```
while [[ $# -gt 0 ]]; do
    case "$1" in
        --test-class) TEST_CLASS="$2"; shift 2 ;;
        --avds) AVDS="$2"; shift 2 ;;
        --evidence-dir) EVIDENCE_DIR="$2"; shift 2 ;;
        --tag) TAG_OVERRIDE="$2"; shift 2 ;;
        --boot-timeout) BOOT_TIMEOUT="$2"; shift 2 ;;
        --concurrent) CONCURRENT="$2"; shift 2 ;;
        --dev) DEV_MODE=1; shift ;;
        --no-build) BUILD_APK=0; shift ;;
        --help|-h)
            cat <<USAGE
Usage: $0 [--test-class <fqcn>] [--avds <list>] [--evidence-dir
        <path>]
          [--tag <tag>] [--boot-timeout <duration>] [--concurrent
        N] [--dev]
          [--no-build]

Defaults:
  --test-class    $DEFAULT_TEST_CLASS
  --avds          $DEFAULT_AVDS
  --evidence-dir  auto: .lava-ci-evidence/Lava-Android-<v>-<vc>/
        matrix/<UTC>/
  --tag           (overrides the auto-detected Lava-Android-<v>-
        <vc> prefix)
  --boot-timeout  5m (forwarded to cmd/emulator-matrix; e.g. 10m,
        600s)
  --concurrent    1 (serial; >1 sets gating=false in the
        attestation)
  --dev           false (set true to permit snapshot reload; sets
        gating=false)

Evidence-path resolution priority:
  1. --evidence-dir <path>     (existing flag, wins)
  2. --tag <tag>               (.lava-ci-evidence/<tag>/matrix/
        <UTC>/)
  3. auto-detect               (.lava-ci-evidence/Lava-Android-
        <v>-<vc>/matrix/<UTC>/)

The AVD list is comma-separated. Each entry MAY include the API
        level
and form factor as Name:APILevel:FormFactor.
USAGE
            exit 0
            ;;
        *) echo "Unknown option: $1"; exit 2 ;;
    esac
done

# Resolve evidence dir if the operator did not pass --evidence-
        dir.
```

```
if [[ -z "$EVIDENCE_DIR" ]]; then
    if [[ -n "$TAG_OVERRIDE" ]]; then
        EVIDENCE_DIR=".lava-ci-evidence/${TAG_OVERRIDE}/matrix/$
        (date -u +%Y-%m-%dT%H-%M-%SZ)"
    else
        prefix=$(detect_version_prefix)
        EVIDENCE_DIR=".lava-ci-evidence/${prefix}/matrix/$(date
        -u +%Y-%m-%dT%H-%M-%SZ)"
    fi
fi
```

- [ ] **Step 3: Forward `--concurrent` and `--dev` to `cmd/emulator-matrix`**

Find the `extra_args=()` block (currently around lines 134-137). Replace with:

```
extra_args=()
if [[ -n "$BOOT_TIMEOUT" ]]; then
    extra_args+=(--boot-timeout "$BOOT_TIMEOUT")
fi
if [[ -n "$CONCURRENT" ]]; then
    extra_args+=(--concurrent "$CONCURRENT")
fi
if [[ "$DEV_MODE" -eq 1 ]]; then
    extra_args+=(--dev)
fi
```

- [ ] **Step 4: Smoke-test the script's --help output**

```
bash scripts/run-emulator-tests.sh --help 2>&1 | head -30
```

Expected: usage block lists `--tag`, `--concurrent N`, `--dev`, and the new evidence-dir resolution priority.

- [ ] **Step 5: Smoke-test the auto-detect — confirm it returns a non-empty prefix**

```
bash -c 'source <(grep -A 30 "^detect_version_prefix()" scripts/
        run-emulator-tests.sh); PROJECT_DIR="$(pwd)"
        detect_version_prefix'
```

Expected: a string like `Lava-Android-1.2.1-127` (matching whatever `app/build.gradle.kts` currently declares) — NOT `Lava-Android-unknown`.

If the output is `Lava-Android-unknown`, inspect `app/build.gradle.kts` and adjust the regex if Lava's version declaration uses single quotes / parentheses / = syntax instead of `versionName "X.Y.Z"`.

Task B2: scripts/tag.sh — 3 new gates

Files:

- Modify: `scripts/tag.sh`

- [ ] **Step 1: Add a helper that walks every matrix attestation and asserts the 3 new gates**

Find the existing `require_matrix_attestation_clause_6_I` helper (starts around line 290) and append a new helper IMMEDIATELY AFTER it (just below its closing `}`). Locate the closing `}` of that helper first, then insert:

```
# Group B clause 6.I extension — three additional gates on every
# matrix attestation. Asserts:
#
#   Gate 1: no row carries concurrent != 1
#           (developer-iteration evidence cannot gate a tag)
#   Gate 2: run-level gating == true
#           (run was --concurrent or --dev)
#   Gate 3: per-row diag.sdk == row.api_level
#           (the "AVD shadow" bluff — claimed API-N row but the
#            running emulator reported a different SDK)
#
# Each gate is a hard die() — tag.sh refuses the tag and prints
       the
# violating rows.
require_matrix_attestation_group_b_gates() {
  local tag_id="$1" pack_dir="$2"

  local files
  mapfile -t files < <(find "$pack_dir" -type f -name 'real-
      device-verification.json' 2>/dev/null)
  if (( ${#files[@]} == 0 )); then
    # Already handled by require_matrix_attestation_clause_6_I; do
    # not double-die. Group B gates only apply when at least one
    # attestation exists.
    return 0
  fi

  local f
  for f in "${files[@]}"; do
    # Gate 1 — reject any row whose concurrent != 1.
    local bad_concurrent
    bad_concurrent=$(jq -r '.rows[] | select(.concurrent != null
        and .concurrent != 1) | "\(.avd) (concurrent=\
        (.concurrent))"' "$f" 2>/dev/null)
    if [[ -n "$bad_concurrent" ]]; then
      die "Cannot tag $tag_id: clause 6.I Group B Gate 1 —
        attestation $f has rows with concurrent != 1 (developer-
        iteration evidence cannot gate a tag): $bad_concurrent"
```

```
    fi

    # Gate 2 — reject if run-level gating is anything other than
        true.
    # Older attestations (pre-Group-B) lack the field; treat
        absent as
    # true to remain backward-compatible with already-shipped
        evidence
    # under .lava-ci-evidence/. This is the ONLY backward-compat
    # carve-out — once Group B ships, every new attestation MUST
        carry
    # the field.
    local gating
    gating=$(jq -r '.gating // "true"' "$f" 2>/dev/null)
    if [[ "$gating" != "true" ]]; then
      die "Cannot tag $tag_id: clause 6.I Group B Gate 2 —
        attestation $f has gating: $gating (run was --concurrent
        or --dev)"
    fi

    # Gate 3 — reject if any row's diag.sdk does not equal its
    # api_level. Skip rows that lack diag (older attestations) for
    # the same backward-compat reason as Gate 2.
    local mismatches
    mismatches=$(jq -r '.rows[] | select(.diag.sdk != null
        and .api_level != null and .diag.sdk != .api_level) | "\
        (.avd): claimed api_level=\(.api_level) but diag.sdk=\
        (.diag.sdk)"' "$f" 2>/dev/null)
    if [[ -n "$mismatches" ]]; then
      die "Cannot tag $tag_id: clause 6.I Group B Gate 3 —
        attestation $f has rows whose diag.sdk does not match
        api_level (the AVD-shadow bluff):
$mismatches"
    fi
  done

  log "[android] Group B clause 6.I gates OK across ${#files[@]}
        attestation file(s)"
}
```

- [ ] **Step 2: Wire the helper into the existing flow**

Find the line in `require_evidence_for_android` that invokes `require_matrix_attestation_clause_6_I "$tag_id" "$pack_dir"` (around line 278). Add a second invocation immediately after:

```
  require_matrix_attestation_clause_6_I "$tag_id" "$pack_dir"
  require_matrix_attestation_group_b_gates "$tag_id" "$pack_dir"
```

- [ ] **Step 3: Confirm the script still parses + the help works**

```
bash -n scripts/tag.sh
bash scripts/tag.sh --help 2>&1 | head -20
```

Expected: `bash -n` exits 0 (no syntax errors); the help block prints.

Task B3: tag-helper tests — failing fixtures first

Files:

- Create: `tests/tag-helper/run_all.sh`
- Create: `tests/tag-helper/test_tag_rejects_concurrent_attestation.sh`
- Create: `tests/tag-helper/test_tag_rejects_non_gating_attestation.sh`
- Create: `tests/tag-helper/test_tag_rejects_diag_sdk_mismatch.sh`
- Create: `tests/tag-helper/test_tag_accepts_gating_serial_attestation.sh`

☐ **Step 1: Create the test runner**

Create `tests/tag-helper/run_all.sh`:

```
#!/usr/bin/env bash
# tests/tag-helper/run_all.sh — execute every test_tag_*.sh in this
# directory; exit non-zero if any fail.
set -u
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
fails=0
total=0
for t in "$SCRIPT_DIR"/test_tag_*.sh; do
  total=$((total + 1))
  if bash "$t"; then
    echo "[tag-helper] PASS: $(basename "$t")"
  else
    echo "[tag-helper] FAIL: $(basename "$t")"
    fails=$((fails + 1))
  fi
done
echo "[tag-helper] $((total - fails))/$total passed"
exit $fails
```

☐ **Step 2: Create the shared fixture helpers in each test**

Each test creates its own throwaway repo + evidence pack + invokes `scripts/tag.sh`. Create `tests/tag-helper/test_tag_rejects_concurrent_attestation.sh`:

```
#!/usr/bin/env bash
# Asserts: tag.sh refuses an evidence pack whose any row has
        concurrent != 1.
set -u
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "$SCRIPT_DIR/../.." && pwd)"

WORK=$(mktemp -d)
trap 'rm -rf "$WORK"' EXIT

# Seed a minimal Lava-shaped tree under WORK so tag.sh's REPO_ROOT
# detection points at WORK, not the real repo.
cp -r "$REPO_ROOT/scripts" "$WORK/scripts"
mkdir -p "$WORK/buildSrc/src/main/kotlin/lava/conventions"
cat > "$WORK/buildSrc/src/main/kotlin/lava/conventions/
        AndroidCommon.kt" <<'KOTLIN'
package lava.conventions
fun whatever() {
  val compileSdk = 36
}
KOTLIN
mkdir -p "$WORK/app"
cat > "$WORK/app/build.gradle.kts" <<'GRADLE'
android {
  defaultConfig {
    versionName "1.2.1"
    versionCode 127
  }
}
GRADLE
mkdir -p "$WORK/.lava-ci-evidence"

# git-init so tag.sh's git rev-parse works
( cd "$WORK" && git init -q && git config user.email t@t && git
        config user.name t && git add -A && git commit -qm seed )

# Build the evidence pack with a concurrent: 4 row.
PACK="$WORK/.lava-ci-evidence/Lava-Android-1.2.1-127"
mkdir -p "$PACK/challenges" "$PACK/bluff-audit" "$PACK/mirror-
        smoke" "$PACK/matrix/run1"
echo '{}' > "$PACK/ci.sh.json"
for i in 1 2 3 4 5 6 7 8; do
  echo '{"status":"VERIFIED"}' > "$PACK/challenges/C${i}.json"
done
echo '{}' > "$PACK/bluff-audit/x.json"
echo '{}' > "$PACK/mirror-smoke/x.json"
cat > "$PACK/real-device-verification.md" <<'EOF'
status: VERIFIED
EOF

cat > "$PACK/matrix/run1/real-device-verification.json" <<'EOF'
{
  "started_at": "2026-05-06T00:00:00Z",
```

```
  "finished_at": "2026-05-06T00:01:00Z",
  "all_passed": true,
  "gating": false,
  "rows": [
    {
      "avd": "CZ_API28_Phone", "api_level": 28, "form_factor":
        "phone",
      "test_class": "lava.app.X", "test_passed": true,
      "diag": {"sdk": 28, "device": "Pixel"},
      "failure_summaries": [],
      "concurrent": 4
    }
  ]
}
EOF

# Run tag.sh in a way that triggers the evidence gate. Use --dry-
        run-off
# (do NOT pass --no-evidence-required). Ask for the android tag at
# 1.2.1-127 with no actual git work — we only care about the gate.
out=$(cd "$WORK" && bash scripts/tag.sh --app android --no-bump --
        no-push --version 1.2.1 --version-code 127 2>&1) && rc=0
        || rc=$?

if [[ $rc -eq 0 ]]; then
  echo "FAIL: tag.sh exited 0 despite concurrent != 1 row"
  echo "$out"
  exit 1
fi
if ! grep -q "Group B Gate 1" <<<"$out"; then
  echo "FAIL: tag.sh exited non-zero but for the wrong reason"
  echo "$out"
  exit 1
fi
exit 0
```

- [ ] **Step 3: Create the non-gating-rejection test**

Create `tests/tag-helper/test_tag_rejects_non_gating_attestation.sh`:

```
#!/usr/bin/env bash
# Asserts: tag.sh refuses an evidence pack whose run-level gating
        is false.
set -u
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "$SCRIPT_DIR/../.." && pwd)"

WORK=$(mktemp -d)
trap 'rm -rf "$WORK"' EXIT
cp -r "$REPO_ROOT/scripts" "$WORK/scripts"
mkdir -p "$WORK/buildSrc/src/main/kotlin/lava/conventions"
```

```
cat > "$WORK/buildSrc/src/main/kotlin/lava/conventions/
        AndroidCommon.kt" <<'KOTLIN'
package lava.conventions
fun whatever() {
  val compileSdk = 36
}
KOTLIN
mkdir -p "$WORK/app"
cat > "$WORK/app/build.gradle.kts" <<'GRADLE'
android { defaultConfig { versionName "1.2.1"; versionCode 127 } }
GRADLE
mkdir -p "$WORK/.lava-ci-evidence"
( cd "$WORK" && git init -q && git config user.email t@t && git
        config user.name t && git add -A && git commit -qm seed )

PACK="$WORK/.lava-ci-evidence/Lava-Android-1.2.1-127"
mkdir -p "$PACK/challenges" "$PACK/bluff-audit" "$PACK/mirror-
        smoke" "$PACK/matrix/run1"
echo '{}' > "$PACK/ci.sh.json"
for i in 1 2 3 4 5 6 7 8; do echo '{"status":"VERIFIED"}' >
        "$PACK/challenges/C${i}.json"; done
echo '{}' > "$PACK/bluff-audit/x.json"
echo '{}' > "$PACK/mirror-smoke/x.json"
cat > "$PACK/real-device-verification.md" <<'EOF'
status: VERIFIED
EOF

cat > "$PACK/matrix/run1/real-device-verification.json" <<'EOF'
{
  "started_at": "2026-05-06T00:00:00Z",
  "finished_at": "2026-05-06T00:01:00Z",
  "all_passed": true,
  "gating": false,
  "rows": [
    {
      "avd": "CZ_API28_Phone", "api_level": 28, "form_factor":
        "phone",
      "test_class": "lava.app.X", "test_passed": true,
      "diag": {"sdk": 28, "device": "Pixel"},
      "failure_summaries": [], "concurrent": 1
    }
  ]
}
EOF

out=$(cd "$WORK" && bash scripts/tag.sh --app android --no-bump --
        no-push --version 1.2.1 --version-code 127 2>&1) && rc=0
        || rc=$?
if [[ $rc -eq 0 ]]; then
  echo "FAIL: tag.sh exited 0 despite gating: false"
  echo "$out"; exit 1
fi
if ! grep -q "Group B Gate 2" <<<"$out"; then
  echo "FAIL: tag.sh exited non-zero but for the wrong reason"
```

```
  echo "$out"; exit 1
fi
exit 0
```

☐

Step 4: Create the diag-sdk-mismatch test

Create `tests/tag-helper/test_tag_rejects_diag_sdk_mismatch.sh`:

```
#!/usr/bin/env bash
# Asserts: tag.sh refuses an evidence pack whose any row has
        diag.sdk != api_level.
set -u
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "$SCRIPT_DIR/../.." && pwd)"

WORK=$(mktemp -d)
trap 'rm -rf "$WORK"' EXIT
cp -r "$REPO_ROOT/scripts" "$WORK/scripts"
mkdir -p "$WORK/buildSrc/src/main/kotlin/lava/conventions"
cat > "$WORK/buildSrc/src/main/kotlin/lava/conventions/
        AndroidCommon.kt" <<'KOTLIN'
package lava.conventions
fun whatever() {
  val compileSdk = 36
}
KOTLIN
mkdir -p "$WORK/app"
cat > "$WORK/app/build.gradle.kts" <<'GRADLE'
android { defaultConfig { versionName "1.2.1"; versionCode 127 } }
GRADLE
mkdir -p "$WORK/.lava-ci-evidence"
( cd "$WORK" && git init -q && git config user.email t@t && git
        config user.name t && git add -A && git commit -qm seed )

PACK="$WORK/.lava-ci-evidence/Lava-Android-1.2.1-127"
mkdir -p "$PACK/challenges" "$PACK/bluff-audit" "$PACK/mirror-
        smoke" "$PACK/matrix/run1"
echo '{}' > "$PACK/ci.sh.json"
for i in 1 2 3 4 5 6 7 8; do echo '{"status":"VERIFIED"}' >
        "$PACK/challenges/C${i}.json"; done
echo '{}' > "$PACK/bluff-audit/x.json"
echo '{}' > "$PACK/mirror-smoke/x.json"
cat > "$PACK/real-device-verification.md" <<'EOF'
status: VERIFIED
EOF

cat > "$PACK/matrix/run1/real-device-verification.json" <<'EOF'
{
  "started_at": "2026-05-06T00:00:00Z",
  "finished_at": "2026-05-06T00:01:00Z",
  "all_passed": true,
  "gating": true,
```

```
  "rows": [
    {
      "avd": "CZ_API28_Phone", "api_level": 28, "form_factor":
        "phone",
      "test_class": "lava.app.X", "test_passed": true,
      "diag": {"sdk": 28, "device": "Pixel"}, "failure_summaries":
        [], "concurrent": 1
    },
    {
      "avd": "CZ_API34_Phone", "api_level": 34, "form_factor":
        "phone",
      "test_class": "lava.app.X", "test_passed": true,
      "diag": {"sdk": 33, "device": "Pixel"}, "failure_summaries":
        [], "concurrent": 1
    },
    {
      "avd": "CZ_API30_Phone", "api_level": 30, "form_factor":
        "phone",
      "test_class": "lava.app.X", "test_passed": true,
      "diag": {"sdk": 30, "device": "Pixel"}, "failure_summaries":
        [], "concurrent": 1
    },
    {
      "avd": "CZ_API36_Phone", "api_level": 36, "form_factor":
        "phone",
      "test_class": "lava.app.X", "test_passed": true,
      "diag": {"sdk": 36, "device": "Pixel"}, "failure_summaries":
        [], "concurrent": 1
    }
  ]
}
EOF

out=$(cd "$WORK" && bash scripts/tag.sh --app android --no-bump --
        no-push --version 1.2.1 --version-code 127 2>&1) && rc=0
        || rc=$?
if [[ $rc -eq 0 ]]; then
  echo "FAIL: tag.sh exited 0 despite diag.sdk != api_level row"
  echo "$out"; exit 1
fi
if ! grep -q "Group B Gate 3" <<<"$out"; then
  echo "FAIL: tag.sh exited non-zero but for the wrong reason"
  echo "$out"; exit 1
fi
exit 0
```

☐

Step 5: Create the golden-path acceptance test

Create `tests/tag-helper/test_tag_accepts_gating_serial_attestation.sh`:

```
#!/usr/bin/env bash
# Asserts: tag.sh accepts a clean serial gating attestation
        through ALL
# 3 Group B gates. The test passes if tag.sh DOES NOT die with a
# Group B Gate 1/2/3 error. (The clause-6.I clause-7 helper
        requires
# coverage of API levels 28/30/34/compileSdk; we provide all
        four.)
set -u
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "$SCRIPT_DIR/../.." && pwd)"

WORK=$(mktemp -d)
trap 'rm -rf "$WORK"' EXIT
cp -r "$REPO_ROOT/scripts" "$WORK/scripts"
mkdir -p "$WORK/buildSrc/src/main/kotlin/lava/conventions"
cat > "$WORK/buildSrc/src/main/kotlin/lava/conventions/
        AndroidCommon.kt" <<'KOTLIN'
package lava.conventions
fun whatever() {
  val compileSdk = 36
}
KOTLIN
mkdir -p "$WORK/app"
cat > "$WORK/app/build.gradle.kts" <<'GRADLE'
android { defaultConfig { versionName "1.2.1"; versionCode 127 } }
GRADLE
mkdir -p "$WORK/.lava-ci-evidence"
( cd "$WORK" && git init -q && git config user.email t@t && git
        config user.name t && git add -A && git commit -qm seed )

PACK="$WORK/.lava-ci-evidence/Lava-Android-1.2.1-127"
mkdir -p "$PACK/challenges" "$PACK/bluff-audit" "$PACK/mirror-
        smoke" "$PACK/matrix/run1"
echo '{}' > "$PACK/ci.sh.json"
for i in 1 2 3 4 5 6 7 8; do echo '{"status":"VERIFIED"}' >
        "$PACK/challenges/C${i}.json"; done
echo '{}' > "$PACK/bluff-audit/x.json"
echo '{}' > "$PACK/mirror-smoke/x.json"
cat > "$PACK/real-device-verification.md" <<'EOF'
status: VERIFIED
EOF

cat > "$PACK/matrix/run1/real-device-verification.json" <<'EOF'
{
  "started_at": "2026-05-06T00:00:00Z",
  "finished_at": "2026-05-06T00:01:00Z",
  "all_passed": true,
  "gating": true,
  "rows": [
    {"avd":"CZ_API28_Phone","api_level":28,"form_factor":"phone","test_class":"lav
        {"sdk":28},"failure_summaries":[],"concurrent":1},
    {"avd":"CZ_API30_Phone","api_level":30,"form_factor":"phone","test_class":"lav
        {"sdk":30},"failure_summaries":[],"concurrent":1},
```

```
    {"avd":"CZ_API34_Phone","api_level":34,"form_factor":"phone","test_class":"lav
        {"sdk":34},"failure_summaries":[],"concurrent":1},
    {"avd":"CZ_API36_Phone","api_level":36,"form_factor":"phone","test_class":"lav
        {"sdk":36},"failure_summaries":[],"concurrent":1}
  ]
}
EOF

out=$(cd "$WORK" && bash scripts/tag.sh --app android --no-bump --
        no-push --version 1.2.1 --version-code 127 2>&1) && rc=0
        || rc=$?
# We tolerate non-zero exit if the failure is for some non-GroupB
        reason
# (e.g. tag.sh's git-tagging logic later in the script needs an
        actual
# remote); we only fail if a Group B gate misfires.
if grep -qE "Group B Gate [123]" <<<"$out"; then
  echo "FAIL: a Group B gate misfired on a clean attestation"
  echo "$out"; exit 1
fi
exit 0
```

- [ ] **Step 6: Make all the test scripts executable**

```
chmod +x tests/tag-helper/run_all.sh tests/tag-helper/
        test_tag_*.sh
```

- [ ] **Step 7: Run the full tag-helper suite**

```
bash tests/tag-helper/run_all.sh
```

Expected: `4/4 passed` (or `[tag-helper] PASS: ...` for each, exit 0).

If a test fails, inspect its diagnostic output and adjust either the fixture (if the assertion is too strict) or `tag.sh`'s helper (if the gate is buggy). Do NOT commit until all 4 pass.

- [ ] **Step 8: Falsifiability rehearsal — drop Gate 3 in tag.sh**

Temporarily comment out the Gate-3 block in `require_matrix_attestation_group_b_gates`:

```
# Gate 3 — DISABLED FOR REHEARSAL
# local mismatches=...
# if [[ -n "$mismatches" ]]; then ...; fi
```

Run `bash tests/tag-helper/test_tag_rejects_diag_sdk_mismatch.sh`.

Expected: `FAIL: tag.sh exited 0 despite diag.sdk != api_level row` (or `FAIL: tag.sh exited non-zero but for the wrong reason`).

Save the observed failure verbatim for the commit body Bluff-Audit stamp.

Restore the Gate-3 block. Run again — PASS.

- [ ] **Step 9: Run the full local pre-push subset (CI + the existing pre-push tests still work)**

```
bash tests/pre-push/check4_test.sh
bash tests/pre-push/check5_test.sh
bash tests/check-constitution/check_constitution_test.sh
bash tests/tag-helper/run_all.sh
```

Expected: every test passes.

Task B4: CLAUDE.md — §6.I clause 4 row-schema documentation

Files:

- Modify: `CLAUDE.md`

- [ ] **Step 1: Append to the §6.I clause 4 paragraph documenting the new row-schema fields**

Find the §6.I clause-4 description in `CLAUDE.md` (begins with `4. **Per-AVD attestation row.**`). Immediately after the existing sentence ending `... timestamp.`, append the Group B extension:

```
**Group B extension (added 2026-05-05 evening):** every row
        additionally carries `diag` (target/sdk/device/
        adb_devices_state — the per-AVD forensic snapshot captured
        immediately before instrumentation invocation),
        `failure_summaries` (parsed JUnit `<failure>` + `<error>`
        entries; empty array on pass), and `concurrent` (the
        matrix runner's `--
        concurrent` setting at the time the row ran; 1 = serial).
        The run-level `gating` field (boolean; true ⇔ `--
        concurrent == 1` AND `--dev=false`) is the constitutional
        eligibility flag — `scripts/tag.sh` refuses to operate on
        attestations whose `gating` is false OR whose any row has
        `concurrent != 1` OR whose any row's `diag.sdk` does not
        equal `api_level` (the AVD-shadow bluff). These three
        additional gates are tag-time enforcement of the per-row
        evidence the Group B spec at `docs/superpowers/specs/
        2026-05-05-anti-bluff-mandate-reinforcement-group-b-
        design.md` (commit e2a3af2) introduces.
```

- [ ]

Step 2: Confirm CLAUDE.md still parses through `scripts/check-constitution.sh`

```
bash scripts/check-constitution.sh
```

Expected: `OK` / `PASS` overall — the constitution-checker's existing §6.N propagation count still passes, and the new sentence does not break any existing assertion.

If `scripts/check-constitution.sh` fails, inspect the failing section. The most likely cause is the new sentence accidentally containing a phrase the checker greps for as a forbidden pattern — adjust wording.

Task B5: Phase B commit

Files:

- (no edits, just commit)

☐ **Step 1: Stage every Phase B change**

```
cd /run/media/milosvasic/DATA4TB/Projects/Lava
git status
git add scripts/run-emulator-tests.sh scripts/tag.sh \
        tests/tag-helper/run_all.sh \
        tests/tag-helper/
        test_tag_rejects_concurrent_attestation.sh \
        tests/tag-helper/
        test_tag_rejects_non_gating_attestation.sh \
        tests/tag-helper/test_tag_rejects_diag_sdk_mismatch.sh \
        tests/tag-helper/
        test_tag_accepts_gating_serial_attestation.sh \
        CLAUDE.md
git status
```

Expected: 8 files staged; nothing else unstaged.

☐ **Step 2: Run all Lava-side tests one final time**

```
bash tests/pre-push/check4_test.sh
bash tests/pre-push/check5_test.sh
bash tests/check-constitution/check_constitution_test.sh
bash tests/tag-helper/run_all.sh
bash scripts/check-constitution.sh
```

Expected: every test/script exits 0.

☐

Step 3: Commit with the Bluff-Audit stamp

```
git commit -m "$(cat <<'EOF'
feat(group-b): tag-time clause 6.I gates + run-emulator-tests
        pack-dir convention

Lava parent code for Group B (anti-bluff mandate reinforcement,
        second
increment after Group A-prime). Spec: docs/superpowers/specs/
        2026-05-05-anti-bluff-mandate-reinforcement-group-b-
        design.md
(commit e2a3af2). Containers branch: lava-pin/2026-05-06-group-b
(pin-bumped in the next commit per the mandatory sequence).

Three changes:

1. scripts/run-emulator-tests.sh — auto-detect versionName/
        versionCode
   from app/build.gradle.kts; --tag <tag> override; --concurrent N
   and --dev passthrough. Evidence dir resolves as
   .lava-ci-evidence/<prefix>/matrix/<UTC>/ where prefix is
   Lava-Android-<v>-<vc> (auto-detected) OR --tag value (override)
        OR
   --evidence-dir (still wins if explicitly given).

2. scripts/tag.sh — require_matrix_attestation_group_b_gates
        helper
   wired after the existing clause-6.I-clause-7 helper. Three new
   gates per attestation file:
     Gate 1: reject any row with concurrent != 1
     Gate 2: reject run-level gating != true
     Gate 3: reject any row with diag.sdk != api_level
   Older attestations (pre-Group-B) lack these fields; treated as
   backward-compat absent → true / not-checked, with the carve-out
   limited to fields whose absence is benign.

3. tests/tag-helper/ — 4 fixture tests + a runner. Each builds a
   minimal Lava-shaped tree under mktemp, seeds a synthetic
   attestation JSON, runs tag.sh, asserts exit code + error
   substring. Coverage: rejects-concurrent, rejects-non-gating,
   rejects-diag-sdk-mismatch, accepts-clean-gating-serial.

Bluff-Audit:

  Test:     test_tag_rejects_diag_sdk_mismatch.sh
  Mutation: comment out the Gate-3 mismatches block in
            require_matrix_attestation_group_b_gates
  Observed: FAIL: tag.sh exited 0 despite diag.sdk != api_level
        row
  Reverted: yes

Runtime evidence:

  bash tests/tag-helper/run_all.sh           → 4/4 passed
```

```
  bash tests/pre-push/check4_test.sh         → ok
  bash tests/pre-push/check5_test.sh         → ok
  bash tests/check-constitution/check_constitution_test.sh → ok
  bash scripts/check-constitution.sh         → ok

CLAUDE.md §6.I clause 4 paragraph extended documenting diag /
failure_summaries / concurrent row fields and the run-level gating
flag, including the three tag.sh gates.

Constitutional bindings: §6.I (matrix-runner is the gate), §6.J /
        §6.L
(anti-bluff functional-reality mandate), §6.N.1.2 (per-matrix-
        runner /
gate change → Bluff-Audit stamp; pre-push Check 4 enforces it).

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

- [ ] **Step 4: Push to all 4 upstreams + verify SHA convergence**

```
for r in github gitlab gitflic gitverse; do
  echo "=== $r ==="
  git push "$r" master
done
echo
for r in github gitlab gitflic gitverse; do
  sha=$(git ls-remote "$r" refs/heads/master 2>/dev/null | awk
        '{print $1}')
  echo "$r: $sha"
done
```

Expected: 4 pushes succeed; identical SHA from all 4 remotes.

Phase C — Pin bump + bluff-hunt + closure attestation (1 commit on `master`)

Task C1: Bump the Containers pin

Files:

- Modify: `submodules/containers` (gitlink)

- [ ] **Step 1: Update the submodule's working tree to the new branch HEAD**

```
cd /run/media/milosvasic/DATA4TB/Projects/Lava/submodules/
        containers
git fetch github lava-pin/2026-05-06-group-b
git checkout lava-pin/2026-05-06-group-b
git pull github lava-pin/2026-05-06-group-b
git rev-parse HEAD
```

Save the printed SHA (call it `$CONTAINERS_HEAD`).

- [ ] **Step 2: Stage the gitlink update in the Lava parent**

```
cd /run/media/milosvasic/DATA4TB/Projects/Lava
git status   # should show: modified: submodules/containers (new
        commits)
git diff --submodule=log submodules/containers
```

Expected: the diff shows the new commits the pin bump will pick up.

Task C2: bluff-hunt JSON

Files:

- Create: `.lava-ci-evidence/bluff-hunt/2026-05-06-group-b.json`

- [ ] **Step 1: Pick 1-2 production files from gate-shaping surface**

Per §6.N.1.1 subsequent-same-day rule, a 1-2-file lighter incident-response hunt. Targets: `cleanup.go::KillByPort` (1 file from Group B gate-shaping surface) + `matrix.go::parseJUnitFailures` (1 file from Group B gate-shaping surface).

For each target, the hunt records a deliberate mutation, the test that caught it, and the observed failure message — the same mutation rehearsals the Phase A commit body already documents (so the JSON cross-references the commit, not duplicates work).

- [ ] **Step 2: Write the JSON**

Create `.lava-ci-evidence/bluff-hunt/2026-05-06-group-b.json`:

```
{
  "date": "2026-05-06",
  "phase": "group-b-evening-incident-response-hunt",
  "rule": "§6.N.1.1 subsequent-same-day lighter hunt — 1-2
        production-code files from gate-shaping surface",
  "targets": [
    {
```

```
      "file": "submodules/containers/pkg/emulator/cleanup.go",
      "function": "KillByPort / killByPortWithDeps",
      "mutation": "weaken the strict adjacent-token matcher to
        strings.Contains(token, target)",
      "covering_test": "submodules/containers/pkg/emulator/
        cleanup_test.go::TestKillByPort_SubstringSafety",
      "observed_failure": "expected Matched=0 (no adjacent token
        pair), got 2 (signaled=...)",
      "reverted": true,
      "commit_reference": "Containers branch lava-pin/2026-05-06-
        group-b — see commit body Bluff-Audit stamp"
    },
    {
      "file": "submodules/containers/pkg/emulator/matrix.go",
      "function": "parseJUnitFailures",
      "mutation": "comment out the for _, e := range tc.Errors
        loop in parseJUnitFailures",
      "covering_test": "submodules/containers/pkg/emulator/
        matrix_test.go::TestParseJUnitFailures_FailureAndError_BothCaptured",
      "observed_failure": "expected 2 entries (1 failure + 1
        error), got 1",
      "reverted": true,
      "commit_reference": "Containers branch lava-pin/2026-05-06-
        group-b — see commit body Bluff-Audit stamp"
    }
  ],
  "outcome": "no surviving bluffs found; both targets fail under
        deliberate mutation as expected",
  "next_phase_prep":
        "Group C scope candidates: network simulation per AVD,
        hardware-screenshot capture per row, AVD image-cache
        management"
}
```

Task C3: Closure attestation

Files:

- Create: `.lava-ci-evidence/Phase-Group-B-closure-2026-05-06.json`

☐ **Step 1: Capture the SHAs needed for the closure record**

```
cd /run/media/milosvasic/DATA4TB/Projects/Lava
LAVA_PHASE_B_SHA=$(git log -1 --format=%H -- scripts/tag.sh)
echo "Lava parent Phase B SHA: $LAVA_PHASE_B_SHA"

cd submodules/containers
CONTAINERS_PHASE_A_SHA=$(git rev-parse HEAD)
echo "Containers Phase A SHA: $CONTAINERS_PHASE_A_SHA"

cd /run/media/milosvasic/DATA4TB/Projects/Lava
```

```
for r in github gitlab gitflic gitverse; do
  echo -n "lava parent on $r: "
  git ls-remote "$r" refs/heads/master 2>/dev/null | awk '{print
        $1}'
done
echo
cd submodules/containers
for r in github gitlab gitflic gitverse; do
  echo -n "containers branch on $r: "
  git ls-remote "$r" refs/heads/lava-pin/2026-05-06-group-b 2>/
        dev/null | awk '{print $1}'
done
```

Save every SHA from the above.

- [ ] **Step 2: Write the closure JSON**

Create `.lava-ci-evidence/Phase-Group-B-closure-2026-05-06.json`. Substitute the SHAs you captured above for the `<...>` placeholders:

```
{
  "date": "2026-05-06",
  "spec": "docs/superpowers/specs/2026-05-05-anti-bluff-mandate-
        reinforcement-group-b-design.md",
  "spec_commit": "e2a3af2",
  "plan": "docs/superpowers/plans/2026-05-05-anti-bluff-mandate-
        reinforcement-group-b.md",
  "branches": {
    "containers": "lava-pin/2026-05-06-group-b",
    "lava_parent": "master"
  },
  "commits": {
    "containers_phase_a": "<CONTAINERS_PHASE_A_SHA>",
    "lava_phase_b": "<LAVA_PHASE_B_SHA>",
    "lava_phase_c": "<filled in by the commit-and-amend below>"
  },
  "components_implemented": {
    "A_KillByPort":
        "submodules/containers/pkg/emulator/cleanup.go (+
        cleanup_test.go) — strict adjacent /proc walk; 4 tests
        including TestKillByPort_SubstringSafety",
    "B_Teardown_FastPath": "submodules/containers/pkg/emulator/
        android.go (+ android_test.go) — post-30s-grace KillByPort
        fast-path; skip-on-mismatch; 2 tests",
    "C_Types": "submodules/containers/pkg/emulator/types.go —
        DiagnosticInfo, FailureSummary, KillReport,
        MatrixResult.Gating, MatrixConfig.Concurrent + .Dev,
        TestResult.Diag + .FailureSummaries + .Concurrent",
    "D_Matrix": "submodules/containers/pkg/emulator/matrix.go —
        runOne extracted, captureDiagnostic, parseJUnitFailures,
        worker pool, writeAttestation new fields + 8 tests",
```

```
    "E_CLI": "submodules/containers/cmd/emulator-matrix/main.go —
        --concurrent N + --dev flags + Gating-status summary
        line",
    "F_RunEmulatorTests": "scripts/run-emulator-tests.sh — auto-
        detect versionName/versionCode + --tag override + --
        concurrent + --dev passthrough + new evidence-dir
        convention",
    "G_TagSh": "scripts/tag.sh —
        require_matrix_attestation_group_b_gates helper (Gate 1
        reject concurrent != 1, Gate 2 reject gating != true, Gate
        3 reject diag.sdk != api_level)",
    "H_Tests_Evidence": "tests/tag-helper/{run_all.sh + 4
        test_tag_*.sh fixtures} + .lava-ci-evidence/bluff-hunt/
        2026-05-06-group-b.json + this closure file"
  },
  "mutation_rehearsals": [
    {
      "test": "TestKillByPort_SubstringSafety",
      "mutation": "weaken matcher to strings.Contains(token,
        target)",
      "observed": "expected Matched=0 (no adjacent token pair),
        got 2",
      "reverted": true
    },
    {
      "test": "TestTeardown_FastPath_SkipsOnMismatch",
      "mutation": "drop the if report.Matched == 0 early-return
        guard",
      "observed": "expected Teardown to return an error when
        KillByPort.Matched==0 and emulator persists, got nil",
      "reverted": true
    },
    {
      "test":
        "TestParseJUnitFailures_FailureAndError_BothCaptured",
      "mutation": "comment out the for _, e := range tc.Errors
        loop",
      "observed": "expected 2 entries (1 failure + 1 error), got
        1",
      "reverted": true
    },
    {
      "test": "TestRunMatrix_Gating_TrueOnDefaults",
      "mutation":
        "change `Gating: concurrent == 1 && !config.Dev` to
        `Gating: false`",
      "observed": "expected Gating=true on defaults (serial, non-
        dev), got false",
      "reverted": true
    },
    {
      "test": "test_tag_rejects_diag_sdk_mismatch.sh",
      "mutation": "comment out the Gate-3 mismatches block in
        require_matrix_attestation_group_b_gates",
```

```
      "observed": "FAIL: tag.sh exited 0 despite diag.sdk !=
        api_level row",
      "reverted": true
    }
  ],
  "mirror_convergence": {
    "containers_branch": {
      "github": "<sha>",
      "gitlab": "<sha>",
      "gitflic": "<sha>",
      "gitverse": "<sha>"
    },
    "lava_parent_master_after_phase_b": {
      "github": "<sha>",
      "gitlab": "<sha>",
      "gitflic": "<sha>",
      "gitverse": "<sha>"
    },
    "verification_method": "live git ls-remote (NOT stale .git/
        refs/remotes/<r>/<branch>)"
  },
  "bluff_hunt_reference": ".lava-ci-evidence/bluff-hunt/
        2026-05-06-group-b.json",
  "next_group_preview":
        "Group C scope candidates (forward-looking only; not
        committed): network simulation per AVD, hardware-
        screenshot capture per row, AVD image-cache management.
        The next operator anti-bluff-mandate invocation will set
        the precedence."
}
```

☐ **Step 3: Stage Phase C changes**

```
cd /run/media/milosvasic/DATA4TB/Projects/Lava
git add submodules/containers \
        .lava-ci-evidence/bluff-hunt/2026-05-06-group-b.json \
        .lava-ci-evidence/Phase-Group-B-closure-2026-05-06.json
git status
```

Expected: 3 changes staged (the gitlink update + 2 evidence files).

☐ **Step 4: Commit Phase C**

```
git commit -m "$(cat <<'EOF'
chore(submodules+evidence): bump Containers pin + Group B closure
        evidence

Bumps submodules/containers to lava-pin/2026-05-06-group-b HEAD,
which contains the Group B Phase A code (KillByPort fast-path,
DiagnosticInfo + FailureSummary types, matrix.go runOne + JUnit
parser + worker pool, cmd/emulator-matrix --concurrent + --dev
```

```
flags). Includes the §6.N.1.1 subsequent-same-day bluff-hunt JSON
and the Phase-Group-B closure attestation with per-component SHAs,
5 mutation rehearsals, and 4-mirror convergence verified via live
git ls-remote.

Phase C, no code changes — pin + evidence only. The previous
        parent
commit (Phase B) added scripts/run-emulator-tests.sh + scripts/
        tag.sh
+ tests/tag-helper/* + CLAUDE.md §6.I clause 4 row-schema docs.

Closure file: .lava-ci-evidence/Phase-Group-B-
        closure-2026-05-06.json
Bluff-hunt:   .lava-ci-evidence/bluff-hunt/2026-05-06-group-b.json

Constitutional bindings: §6.I (matrix-runner is the gate; pin bump
ships the gate's new evidence schema), §6.N.1.1 (subsequent-same-
        day
incident-response hunt — 1-2 production-code files), §6.N.1.3
(per-attestation-file falsifiability rehearsal — recorded inline
        in
the closure JSON).

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

- [ ] **Step 5: Capture the Phase C SHA and amend the closure JSON to include it**

```
LAVA_PHASE_C_SHA=$(git rev-parse HEAD)
echo "Lava Phase C SHA: $LAVA_PHASE_C_SHA"
sed -i "s|\"lava_phase_c\": \"<filled in by the commit-and-amend
        below>\"|\"lava_phase_c\": \"$LAVA_PHASE_C_SHA\"|" \
  .lava-ci-evidence/Phase-Group-B-closure-2026-05-06.json
git add .lava-ci-evidence/Phase-Group-B-closure-2026-05-06.json
git commit --amend --no-edit
git log -1 --format=%H
```

Expected: the new HEAD SHA is different from `$LAVA_PHASE_C_SHA` (because of the amend); update the closure JSON one more time with the new SHA, OR simply leave it referencing the original Phase C commit content as the source of truth and accept that the file's `lava_phase_c` records the parent-of-amend.

(If the amended SHA matters for any later step, just re-run the same `sed` + `commit --amend --no-edit` cycle until the SHA in the file equals `git rev-parse HEAD`. Two iterations suffice.)

- [ ]

Step 6: Push Phase C to all 4 upstreams + verify convergence

```
for r in github gitlab gitflic gitverse; do
  echo "=== $r ==="
  git push "$r" master
done
echo
for r in github gitlab gitflic gitverse; do
  sha=$(git ls-remote "$r" refs/heads/master 2>/dev/null | awk
        '{print $1}')
  echo "$r: $sha"
done
```

Expected: 4 pushes succeed; identical SHA from all 4 remotes.

Final acceptance

- [ ] **All 3 commits land**
  - Containers `lava-pin/2026-05-06-group-b` HEAD = Phase A commit
  - Lava `master` head -1 = Phase C commit (pin bump + evidence)
  - Lava `master` head -2 = Phase B commit (parent code)
- [ ] **All 4 mirrors converge after each push** (via live `git ls-remote`)
- [ ] **All test suites green**
  - `cd submodules/containers && go test ./pkg/emulator/... -count=1 -race` → PASS
  - `bash tests/tag-helper/run_all.sh` → 4/4 passed
  - `bash tests/pre-push/check4_test.sh` → ok
  - `bash tests/pre-push/check5_test.sh` → ok
  - `bash tests/check-constitution/check_constitution_test.sh` → ok
  - `bash scripts/check-constitution.sh` → ok
- [ ] **5 mutation rehearsals recorded** (4 in Phase A commit body + 1 in Phase B commit body, plus 2 cross-referenced in the bluff-hunt JSON for the §6.N.1.1 record)
- [ ] **Closure JSON committed** with all SHAs filled in (Phase A, Phase B, Phase C, plus per-mirror SHAs verified via live ls-remote)

Self-Review

This block is the spec-vs-plan reconciliation per the writing-plans skill checklist.

1. Spec coverage:

Spec component	Plan task
A — KillByPort	A1 (failing tests), A2 (impl + falsifiability rehearsal)
B — Teardown fast-path	A3 (failing tests), A4 (impl + falsifiability rehearsal)
C — DiagnosticInfo + KillReport + AVDRow ext	A5
D — diag capture + JUnit parse + worker pool	A6, A7 (parser tests + Gating tests + 2 falsifiability rehearsals)
E — `--concurrent` + `--dev` flags	A8
F — run-emulator-tests.sh evidence-path auto-detect	B1
G — tag.sh 3 new gates	B2
H — tag-helper tests + bluff-hunt + closure attestation	B3 (tests + falsifiability rehearsal), C2 (bluff-hunt), C3 (closure)
Mirror policy + branch policy	"Mirror & branch policy" header + push verifications in A9.5, B5.4, C3.6
CLAUDE.md §6.I clause 4 doc extension	B4
Pin bump	C1
5 mutation rehearsals	A2.5+A4.5+A7.5+A7.6 (4 in Containers) + B3.8 (1 in Lava parent) = 5

No gaps.

2. Placeholder scan:

- "TBD" / "TODO" / "implement later" / "Add appropriate error handling" — none introduced.
- The closure JSON has `<CONTAINERS_PHASE_A_SHA>` / `<LAVA_PHASE_B_SHA>` / `<sha>` placeholders that are filled in by the operator following the explicit `git rev-parse` + `for r in ...; do git ls-remote ...; done` commands in C3.1 + C3.5 — these are runtime

placeholders the engineer fills in, not plan-author handwaves. Acceptable per writing-plans's "exact commands" rule (the commands ARE exact; only the values they produce vary by run).

3. Type consistency:

- `KillReport` fields (`Matched`, `Sigtermed`, `Sigkilled`, `Surviving`) used in A1 (test), A2 (impl), A4 (Teardown's report variable) — consistent.
- `DiagnosticInfo` fields (`Target`, `SDK`, `Device`, `ADBDevicesState`) used in A5 (declaration), A6 (`captureDiagnostic` populator), A7 (Gate-3 test references `diag.sdk`), B2 (Gate-3 jq query reads `.diag.sdk`) — consistent.
- `MatrixResult.Gating` declared in A5, populated in A6 (`Gating: concurrent == 1 && !config.Dev`), serialized in A6's `writeAttestation` (`"gating": r.Gating`), tested in A7 (3 Gating tests), checked in B2 (Gate 2 jq query reads `.gating`) — consistent.
- `MatrixConfig.Concurrent` + `.Dev` declared in A5, consumed in A6's `RunMatrix` (concurrent path), wired through CLI flags in A8 — consistent.
- `killByPortHook` + `teardownGracePeriod` package-level vars added in A4; tests in A3 reference them — order is "test references → impl declares" which Go tolerates because tests live in the same package, and the test step explicitly says "compile error citing the two undefined identifiers" expected in A3.2 (proving the test depends on the impl, not the other way around).
- `parseJUnitFailures` declared in A6, referenced in `runOne` (in A6) and tested in A7 — consistent.
- `runOne`, `captureDiagnostic`, `parseAVDTarget`, `maxInt` helpers — all declared in A6 and used only within matrix.go, no external references.

No inconsistencies found.

Execution Handoff

Plan complete and saved to `docs/superpowers/plans/2026-05-05-anti-bluff-mandate-reinforcement-group-b.md`. Two execution options:

1. Subagent-Driven (recommended) — Fresh subagent per task with two-stage review (spec compliance + code quality) between tasks. Same pattern Group A-prime ran successfully.

2. Inline Execution — Execute tasks in this session using `superpowers:executing-plans`, batch execution with checkpoints.

Which approach?